

**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
"НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
"ВЫСШАЯ ШКОЛА ЭКОНОМИКИ"
ФАКУЛЬТЕТ КОМПЬЮТЕРНЫХ НАУК**

Никитюк Михаил Андреевич

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

**Разработка и внедрение комплексного сервиса для управления бронированием
ассистентов и автоматической поддержки студентов с использованием RAG
технологий**

по направлению подготовки 09.04.02 Прикладная математика и информатика
образовательная программа «Аналитика больших данных»

Студент

М. А. Никитюк

Научный руководитель

Академический руководитель

ОП:Аналитика больших данных

Е. А. Паточенко

Москва 2026

Содержание

Аннотация	4
1 Введение	5
2 Теоретическое обоснование принципа работы программы и обзор существующих решений	12
2.1 Введение в проблему автоматизации бронирования ассистентов	12
2.2 Обзор существующих решений и их ограничений	14
2.3 Обоснование выбора разрабатываемого подхода	18
2.4 Выводы по первой главе	21
3 Постановка задачи и требования к системе	23
3.1 Цель и задачи разработки	24
3.2 Описание ролей пользователей	25
3.3 Функциональные требования	26
3.4 Нефункциональные требования	28
3.5 Требования к RAG-помощнику	29
3.6 Выводы по второй главе	30
4 Проектирование архитектуры системы	31
4.1 Общая архитектура сервиса	31
4.2 Структура базы данных	35
4.3 Архитектура клиентской части	37
4.4 Архитектура серверной части основного приложения	40
4.5 Архитектура RAG-модуля	43
4.6 Развёртывание системы	47
4.7 Выводы по третьей главе	49
5 Тестирование и оценка разработанного решения	50
5.1 Подход к тестированию	50
5.2 Проверка пользовательских сценариев	51
5.3 Оценка работы RAG-помощника	52
5.4 Сравнение с исходным процессом на основе таблиц	54
5.5 Сравнение с существующими аналогами	54

5.6 Выводы по четвёртой главе	56
6 Направления дальнейшего развития	58
Заключение	59
Список литературы	61
Приложения	61

Аннотация

В данной работе рассматривается разработка и внедрение комплексного сервиса для управления бронированием ассистентов и автоматической поддержки студентов с использованием RAG-технологий. Актуальность работы связана с тем, что подбор ассистентов, согласование заявок, оформление документов и поиск информации по внутренним регламентам часто выполняются вручную или с помощью больших табличных файлов. Такой подход усложняет работу преподавателей, студентов и менеджеров, повышает риск ошибок и создает административную нагрузку.

Целью работы является создание веб-сервиса, который заменяет разрозненные таблицы единой системой с разграничением пользовательских ролей и встроенным интеллектуальным помощником. В рамках проекта выделены три основные роли: преподаватель, студент и менеджер. Для каждой роли определены основные сценарии взаимодействия, после чего реализованы соответствующие интерфейсы и серверная логика.

Сервис построен на клиент-серверной архитектуре. Клиентская часть реализована с использованием `React` и `Tailwind CSS`, серверная часть основного веб-приложения построена на базе `Node.js` и `Fastify 5`, а для хранения данных используется `PostgreSQL`. Отдельный сервер и вычисления RAG-помощника реализованы на языке `Python`. Для развертывания применяются `Docker` и `Docker Compose`. RAG-помощник работает на основе локальной языковой модели и формирует ответы с опорой на внутренние положения, инструкции по оплатам, регламенты бронирования, шаблоны договоров и FAQ.

В результате был разработан прототип системы, объединяющий бронирование ассистентов, управление заявками, роль менеджера и автоматический поиск ответов по документам. Полученное решение может использоваться для снижения нагрузки на менеджеров, повышения прозрачности процессов и упрощения взаимодействия между участниками образовательного процесса.

Ключевые слова

RAG-технологии, веб-приложение, бронирование ассистентов, `PostgreSQL`, `React`, `Node.js`, `Fastify 5`, образовательные сервисы, автоматизация административных процессов.

1 Введение

Образовательная программа состоит не только из занятий, лекций и семинаров. Вокруг каждого курса есть большое количество сопутствующих процессов: распределение нагрузки, работа с учебными группами, оформление документов, коммуникация между преподавателями, студентами, менеджерами и учебным офисом. Пока участников немного, многие вопросы можно решать вручную, но при росте числа курсов и групп такой подход быстро становится неудобным.

Особенно это заметно в процессах, где одновременно участвуют несколько сторон. Преподавателю важно понимать, кто будет помогать ему на курсе, студенту — какие условия ассистирования и какие действия от него ожидаются, а менеджеру — видеть общую картину и контролировать, что все оформлено корректно. Если информация хранится в разных местах или передается через переписки, появляются задержки, повторные уточнения и ошибки в данных. В итоге даже несложная административная задача начинает занимать больше времени, чем должна.

Поэтому для подобных процессов важно иметь не просто место для хранения информации, а понятный инструмент, который задает общий порядок работы. Такой инструмент должен показывать участникам актуальные данные, разделять действия по ролям и помогать быстрее находить нужные сведения. Без специализированного сервиса процесс становится трудоемким: его сложнее контролировать, сложнее масштабировать и сложнее поддерживать в актуальном состоянии.

На практике подобные процессы часто реализуются с использованием табличных инструментов, таких как Excel или Google Sheets. Несмотря на универсальность и доступность данных решений, их применение для управления бронированием ассистентов имеет ряд существенных ограничений. Таблицы плохо подходят для одновременной работы большого количества пользователей с различными ролями, не обеспечивают удобного разграничения прав доступа, затрудняют контроль статусов заявок и не позволяют в явном виде формализовать бизнес-логику процесса. Кроме того, при увеличении числа преподавателей, студентов и бронирований возрастает риск ошибок, дублирования записей, потери актуальности данных и возникновения дополнительных административных издержек.

Проблема усложняется тем, что взаимодействие между участниками процесса не ограничивается только выбором ассистента. В процессе работы у пользователей регулярно возникают вопросы, связанные с правилами бронирования, порядком оформления документов, механизмом формирования выплат, нормативными ограничениями, сроками подачи заявок

и иными административными аспектами. На практике ответы на такие вопросы редко находятся в одном месте. Часть информации содержится во внутренних положениях, часть - в инструкциях по оплатам, часть - в регламентах бронирования, шаблонах договоров или отдельных списках часто задаваемых вопросов. Из-за этого пользователю приходится либо самостоятельно просматривать несколько документов, либо обращаться к менеджеру за уточнением. При этом сложность состоит не только в объеме документов, но и в том, что пользователь не всегда заранее понимает, в каком именно документе находится нужное правило и какими словами оно сформулировано.

Именно поэтому в рамках данной работы возникла идея не ограничиваться только интерфейсом для бронирования ассистентов, а разработать более комплексный сервис. Такой сервис должен объединять два направления: управление заявками на ассистентов и помощь пользователям при поиске ответов на административные вопросы. В этом случае преподаватель, студент или менеджер получает не просто форму для работы с бронированиями, а единую среду, в которой можно выполнять основные действия и сразу уточнять связанные с ними правила. Для реализации такой поддержки могут использоваться технологии retrieval-augmented generation, далее RAG. Их основная идея заключается в том, что языковая модель получает не только сам вопрос пользователя, но и дополнительные фрагменты из заранее подготовленной базы документов, на которые она может опираться при формировании ответа.

Подход RAG хорошо подходит для задач, где важно отвечать не в общем виде, а с учетом конкретных внутренних материалов организации. Обычный чат-бот может дать правдоподобный ответ, но при этом не всегда понятно, на какой источник он опирается и соответствует ли этот ответ действующим правилам. В RAG-системе перед генерацией ответа выполняется поиск по корпусу документов, после чего найденные фрагменты передаются модели как контекст. Благодаря этому ответ становится более связанным с исходными материалами, а риск появления неточных или слишком общих формулировок снижается. В рассматриваемом проекте такой механизм может использоваться для работы с инструкциями по оплатам, правилами бронирования, шаблонами договоров, FAQ и другими документами, с которыми регулярно сталкиваются преподаватели, студенты и менеджеры.

Актуальность данной работы связана с тем, что ручное сопровождение процесса бронирования ассистентов плохо масштабируется. Пока количество заявок невелико, таблиц и переписка может быть достаточно, однако при росте числа преподавателей, студентов и административных условий такой способ работы становится неудобным. Менеджерам приходится чаще проверять данные, отвечать на повторяющиеся вопросы и следить за тем, чтобы

участники процесса использовали актуальные правила. При этом современные средства веб-разработки и локальные языковые модели позволяют создавать более специализированные решения, ориентированные на конкретную предметную область. В отличие от универсальных сервисов, такая система может учитывать реальные роли пользователей, внутреннюю логику бронирования, особенности хранения данных и требования образовательной организации.

В рамках данной работы рассматривается разработка и внедрение комплексного сервиса для управления бронированием ассистентов и автоматической поддержки студентов с использованием RAG-технологий. Проект является развитием ранее созданного прототипа, предназначенного для замены неудобного процесса работы с большими таблицами. В новой версии системы основное внимание уделяется доработке визуальной части приложения, добавлению роли менеджера, расширению пользовательских сценариев и внедрению RAG-помощника, предназначенного для автоматической поддержки пользователей при работе с внутренними документами.

Разрабатываемая система предполагает наличие трех основных ролей: преподавателя, студента и менеджера. Преподаватель получает возможность создавать и просматривать бронирования ассистентов, выбирать подходящих студентов и отслеживать состояние заявок. Студент получает возможность взаимодействовать с системой как потенциальный ассистент, просматривать релевантную информацию и получать ответы на вопросы, связанные с участием в процессе ассистирования. Менеджер, в свою очередь, получает инструменты для контроля процесса, обработки заявок, сопровождения пользователей и администрирования данных. RAG-помощник в такой структуре выступает как общий инструмент поддержки, доступный для всех трех ролей. Его задача состоит в том, чтобы пользователь мог не искать нужный документ вручную, а задать вопрос в свободной форме и получить ответ, основанный на внутренних инструкциях, положениях и регламентах.

Техническая часть проекта строится вокруг классической клиент-серверной архитектуры. Интерфейс приложения реализован с использованием `React` и `Tailwind CSS`. Такой выбор был сделан из-за того, что данные инструменты позволяют достаточно быстро разрабатывать визуальные компоненты, переиспользовать их в разных частях приложения и адаптировать интерфейс под различные пользовательские сценарии.

Основная серверная часть веб-приложения реализована на `Node.js` с использованием фреймворка `Fastify 5`. Она отвечает за обработку запросов от клиента, выполнение основной бизнес-логики и обмен данными с базой данных. В качестве системы хранения используется `PostgreSQL`, поскольку в проекте необходимо работать со связанными сущностями:

пользователями, ролями, заявками, бронированиями и административными статусами. Для запуска отдельных компонентов системы применяется `Docker` и `Docker Compose`, что упрощает настройку окружения и делает развертывание проекта более воспроизводимым.

Отдельно от основной серверной части реализован модуль RAG-помощника. Он написан на языке `Python`, так как большая часть инструментов для работы с языковыми моделями, обработкой текстов и векторным поиском удобнее доступна именно в этой экосистеме. Вычисления RAG-модуля выполняются с использованием локальной языковой модели. Такой подход был выбран для того, чтобы не передавать внутренние документы во внешние сервисы и сохранить больший контроль над обработкой данных внутри проекта.

Важной особенностью работы является то, что разработка системы выполнялась в условиях необходимости самостоятельного освоения полного цикла веб-разработки. В ходе выполнения проекта были изучены принципы построения клиент-серверных приложений, проектирования интерфейсов, организации серверной логики, работы с базами данных, контейнеризации и интеграции языковых моделей. При изучении отдельных технически сложных конструкций использовались инструменты искусственного интеллекта, которые применялись как вспомогательное средство для объяснения концепций, анализа ошибок и ускорения освоения новых технологий. При этом проектирование, реализация и адаптация системы под предметную область выполнялись автором самостоятельно.

Цель исследования:

Целью данной работы является разработка и внедрение комплексного сервиса для управления бронированием ассистентов и автоматической поддержки пользователей с использованием RAG-технологий, направленного на повышение удобства взаимодействия преподавателей, студентов и менеджеров в рамках образовательного процесса.

Задачи исследования:

1. Проанализировать существующий процесс бронирования ассистентов и выявить основные ограничения использования табличных инструментов для решения данной задачи.
2. Определить роли пользователей системы, их основные сценарии взаимодействия и требования к функциональности сервиса.
3. Разработать архитектуру веб-приложения, включающую клиентскую часть, серверную часть, базу данных и модуль автоматической поддержки пользователей.
4. Реализовать пользовательские интерфейсы для преподавателей, студентов и менеджеров с учетом различий в их задачах и правах доступа.
5. Разработать механизм хранения и обработки данных о пользователях, заявках, бронированиях и связанных административных сущностях.

6. Интегрировать в систему RAG-помощника, использующего внутренние нормативно-правовые положения, инструкции по оплатам, регламенты бронирования, шаблоны договоров и FAQ.

7. Обеспечить возможность развертывания сервиса с использованием `Docker` и `Docker Compose`.

8. Провести анализ разработанного решения с точки зрения удобства использования, полноты реализованной функциональности и применимости в реальном образовательном процессе.

Предмет исследования:

Предметом исследования являются методы и подходы к разработке веб-сервисов для автоматизации административных процессов в образовательной организации, а также применение RAG-технологий для автоматической поддержки пользователей при работе с внутренними документами.

Объект исследования:

Объектом исследования является процесс управления бронированием ассистентов преподавателей, включающий взаимодействие преподавателей, студентов и менеджеров, а также сопутствующие административные процедуры, связанные с оформлением, оплатой и нормативным сопровождением ассистирования.

Методы исследования:

1. Анализ существующего процесса бронирования ассистентов и выявление его основных проблем.

2. Анализ требований пользователей и формализация основных ролей в системе.

3. Проектирование архитектуры клиент-серверного веб-приложения.

4. Проектирование структуры базы данных для хранения информации о пользователях, заявках и бронированиях.

5. Реализация веб-сервиса с использованием `React`, `Node.js`, `Fastify` 5, `Tailwind CSS` и `PostgreSQL`.

6. Интеграция RAG-модуля на основе локальной языковой модели, развернутой на отдельном `Python` сервере.

7. Контейнеризация приложения с использованием `Docker` и `Docker Compose`.

8. Проверка работоспособности системы на основных пользовательских сценариях.

Практическая значимость:

Практическая значимость данной работы заключается в создании сервиса, который может быть использован для упрощения и частичной автоматизации процесса бронирования

ассистентов в образовательной организации. Разработанная система позволяет заменить работу с большими таблицами более структурированным веб-интерфейсом, снизить количество ручных действий со стороны менеджеров и повысить прозрачность процесса для преподавателей и студентов. Внедрение RAG-помощника дополнительно позволяет сократить время, необходимое пользователям для поиска информации во внутренних документах, и уменьшить количество повторяющихся административных обращений.

Теоретическая значимость:

Теоретическая значимость данной работы заключается в рассмотрении подхода к интеграции RAG-технологий в специализированную образовательную информационную систему. В работе анализируется возможность объединения классической клиент-серверной архитектуры, ролевой модели доступа и модуля автоматической поддержки пользователей на основе локальной языковой модели. Полученные результаты могут быть использованы при дальнейшем изучении способов применения языковых моделей в административных процессах образовательных организаций.

Научная новизна данного исследования заключается в следующем:

1. В рамках работы был разработан не отдельный инструмент для закрепления заявок, а комплексный сервис, который объединяет процесс бронирования ассистентов и автоматическую информационную поддержку пользователей.
2. В системе была введена отдельная роль менеджера. Это позволяет перенести административный контроль из внешних таблиц и переписок внутрь самого сервиса, а также сделать процесс обработки заявок более структурированным.
3. Для работы с внутренними документами был реализован RAG-помощник, который использует положения, инструкции, регламенты, шаблоны договоров и FAQ как базу знаний для формирования ответов на вопросы пользователей.
4. Была предложена архитектура, в которой основное веб-приложение, база данных и модуль интеллектуальной поддержки разделены на отдельные компоненты и могут запускаться в контейнеризированной среде.
5. В ходе работы была показана возможность разработки full-stack сервиса, ориентированного на конкретный административный процесс образовательной организации, с последующим расширением его функциональности за счет локальной языковой модели.

Таким образом, результат работы заключается не только в реализации веб-интерфейса для бронирования ассистентов, но и в создании более общей системы, которая помогает формализовать административный процесс и уменьшить количество ручных действий со стороны его участников. Разработанный сервис позволяет повысить удобство взаимодействия

между преподавателями, студентами и менеджерами, а также демонстрирует применимость RAG-технологий для поддержки пользователей при работе с внутренними документами организации.

Структура исследования определена введением, главами основной части, заключением, списком использованных источников и приложениями. В первой главе рассматривается предметная область, существующий процесс бронирования ассистентов и ограничения используемых инструментов. Во второй главе формулируются требования к системе и описываются основные пользовательские роли. В третьей главе рассматривается архитектура и реализация разработанного сервиса. В четвертой главе описывается интеграция RAG-помощника и особенности работы с внутренними документами. В пятой главе приводится оценка разработанного решения и анализ его применимости.

2 Теоретическое обоснование принципа работы программы и обзор существующих решений

2.1 Введение в проблему автоматизации бронирования ассистентов

Современная образовательная среда включает большое количество процессов, связанных не только с проведением занятий и контролем знаний, но и с административным сопровождением учебной деятельности. Одним из таких процессов является организация работы ассистентов преподавателей. Ассистенты могут участвовать в проверке домашних заданий, проведении консультаций, сопровождении проектной деятельности, подготовке учебных материалов и решении других задач, возникающих в рамках образовательного процесса. При этом само назначение ассистента на курс или конкретную задачу является не только учебным, но и административным процессом, так как требует согласования между преподавателем, студентом и менеджером образовательной программы.

На первый взгляд задача бронирования ассистентов может показаться достаточно простой, поскольку она сводится к выбору подходящего студента и фиксации соответствующей договоренности. Однако на практике данный процесс содержит множество дополнительных ограничений. Необходимо учитывать доступность студента, его компетенции, интерес к конкретному курсу, требования преподавателя, количество уже взятых задач, сроки согласования, оформление документов, особенности оплаты и внутренние положения образовательной организации. Кроме того, различные участники процесса обладают разными правами и целями: преподаватель заинтересован в поиске подходящего ассистента, студент — в получении понятной информации о возможностях и условиях работы, а менеджер — в контроле корректности всего процесса.

Если в процессе участвует небольшое количество преподавателей и студентов, то такую работу действительно можно какое-то время вести вручную. В этом случае достаточно общей таблицы, нескольких переписок и набора инструкций, к которым участники обращаются при необходимости. Однако по мере роста числа заявок данный способ начинает создавать все больше трудностей. В таблицах появляются дополнительные поля, статусы и комментарии, отдельные записи могут дублироваться, а актуальное состояние заявки становится не всегда очевидным. Менеджеру приходится постоянно проверять, какие данные уже заполнены, какие заявки требуют действия, кому необходимо ответить и какие правила нужно дополнительно пояснить.

Из-за этого появляется потребность в отдельной программной системе, которая будет

не просто хранить список бронирований, а задавать понятный порядок работы с ними. Такая система должна учитывать, что преподаватель, студент и менеджер выполняют разные действия и видят процесс с разных сторон. Для преподавателя важен выбор подходящего ассистента и состояние его заявки, для студента - понимание доступных возможностей и условий участия, для менеджера - контроль корректности данных и сопровождение процесса. Поэтому в сервисе необходимо явно описать роли пользователей, права доступа, статусы бронирований и действия, которые могут выполняться на каждом этапе.

Еще одна сложность связана с тем, что пользователям часто нужна не только сама форма заявки, но и пояснения к административным правилам. Например, преподавателю может быть важно понять, как правильно оформить бронирование, студенту - какие документы нужно подписать, а менеджеру - быстро найти нужный пункт регламента или инструкцию по выплатам. Эти сведения обычно находятся не в одном документе, а распределены между положениями, инструкциями, шаблонами договоров, регламентами и списками часто задаваемых вопросов. Поэтому даже при наличии всех материалов поиск ответа может занимать время, особенно если пользователь не знает, где именно находится нужная информация.

Для решения такой задачи в работе рассматривается применение технологий retrieval-augmented generation, далее RAG. Идея данного подхода состоит в том, что языковая модель при подготовке ответа использует не только собственные внутренние знания, но и фрагменты документов, найденные по запросу пользователя. Такой принцип был описан Льюисом и соавторами, которые рассматривали RAG как объединение параметрической памяти модели и непараметрической памяти в виде внешнего индекса документов [1]. Для рассматриваемого проекта это особенно важно, так как внутренние документы могут изменяться, а ответы помощника должны быть связаны с актуальными инструкциями и регламентами.

В рамках данной работы RAG-помощник используется как дополнительный модуль сервиса, а не как замена менеджера. Его основная задача заключается в том, чтобы снять часть повторяющихся вопросов и ускорить поиск информации. Пользователь может задать вопрос в свободной форме, после чего система подбирает релевантные фрагменты из внутренней базы документов и формирует ответ на их основе. Такой механизм позволяет объединить в одном сервисе работу с бронированиями и справочную поддержку по положениям, инструкциям по оплатам, правилам оформления, шаблонам договоров и FAQ.

Основные термины:

Веб-сервис - программная система, с которой пользователь работает через веб-интерфейс. В данной работе под веб-сервисом понимается приложение для управления бронированием ассистентов и взаимодействия между преподавателями, студентами и менеджерами.

Пользовательская роль - набор прав и доступных действий, который определяет, как именно пользователь взаимодействует с системой. В разрабатываемом сервисе используются три роли: преподаватель, студент и менеджер. Такое разделение необходимо для того, чтобы каждая группа пользователей работала только с теми функциями, которые относятся к ее задачам.

Бронирование ассистента - заявка на привлечение студента к выполнению ассистентских задач. В системе такое бронирование имеет участников, статус, связанные сроки и административные данные, необходимые для дальнейшего сопровождения процесса.

Менеджер - пользователь, который отвечает за административное сопровождение бронирований. В отличие от преподавателя и студента, менеджер имеет расширенные возможности для проверки заявок, контроля статусов и поддержки пользователей.

RAG-помощник - модуль системы, который ищет релевантные фрагменты во внутренних документах и использует их для формирования ответа на вопрос пользователя. В данной работе такой помощник применяется для поддержки пользователей при работе с регламентами, инструкциями и другими внутренними материалами образовательной организации.

Локальная языковая модель - модель, вычисления которой выполняются на собственной инфраструктуре проекта, а не через внешний облачный API. Такой подход может быть особенно важен в случаях, когда система работает с внутренними документами и административной информацией.

2.2 Обзор существующих решений и их ограничений

Перед разработкой специализированного сервиса необходимо рассмотреть существующие программные решения, которые частично пересекаются с поставленной задачей. К таким решениям можно отнести табличные инструменты, системы онлайн-бронирования, low-code платформы, образовательные LMS-системы, сервисы для проверки работ и системы автоматической поддержки пользователей. Несмотря на то, что каждое из этих решений может закрывать отдельную часть проблемы, их использование в исходном виде не позволяет полностью решить задачу управления бронированием ассистентов и автоматической поддержки пользователей.

Наиболее простым и распространенным подходом является использование электронных таблиц. Таблицы удобны тем, что позволяют быстро создать структуру данных, открыть доступ нескольким пользователям и вручную редактировать записи. Однако их ос-

новное преимущество одновременно становится и ограничением. Таблица не задает строгую бизнес-логику процесса, не обеспечивает полноценного жизненного цикла заявки и не отделяет действия преподавателя, студента и менеджера на уровне интерфейса. В результате контроль корректности данных переносится на пользователей, что увеличивает риск ошибок. Кроме того, при большом числе записей таблицы становятся менее удобными для навигации и анализа.

Другой класс решений составляют сервисы онлайн-бронирования и планирования встреч. Например, **Microsoft Bookings**¹ предназначен для планирования и управления встречами, позволяет пользователям выбирать доступное время и интегрируется с календарем Microsoft 365. Аналогичным образом **Calendly**² предоставляет инструменты для автоматизации назначения встреч, публикации доступных временных слотов, настройки типов событий и напоминаний. Такие сервисы хорошо решают задачу выбора времени, но задача бронирования ассистента шире обычного календарного события. В ней необходимо учитывать не только свободный слот, но и образовательный контекст, роль студента, требования преподавателя, административный статус заявки, возможные ограничения по оплате и документы, связанные с ассистированием. Поэтому использование подобных сервисов не позволяет полностью формализовать процесс.

Существуют также платформы для построения внутренних приложений и рабочих процессов, например **Airtable**³. Данный сервис позиционируется как платформа для создания приложений, управления данными и автоматизации рабочих процессов. По сравнению с обычными таблицами такие инструменты предоставляют больше возможностей для настройки представлений, фильтрации данных и автоматизации отдельных действий. Однако они остаются универсальными платформами, которые требуют дополнительной настройки под конкретный процесс. Для рассматриваемой задачи это означает, что значительная часть предметной логики, включая роли пользователей, обработку заявок, интерфейс менеджера и интеграцию RAG-помощника, должна быть спроектирована поверх базовых возможностей платформы. Кроме того, подобные решения не всегда подходят в случаях, когда требуется полный контроль над хранением данных и вычислениями.

Отдельно необходимо рассмотреть образовательные Отдельно необходимо рассмот-

¹Официальный сайт Microsoft Bookings: <https://www.microsoft.com/en-us/microsoft-365/business/scheduling-and-booking-app>, дата обращения: 25.04.2026.

²Официальный сайт Calendly: <https://calendly.com>, дата обращения: 25.04.2026.

³Официальный сайт Airtable: <https://www.airtable.com>, дата обращения: 25.04.2026.

реть образовательные LMS-системы⁴, такие как Moodle⁵ и Canvas⁶. Moodle является открытой системой управления обучением, предназначенной для создания онлайн-курсов и организации образовательного процесса. Canvas также относится к LMS-системам и применяется для организации курсов, размещения заданий, выставления оценок, сбора обратной связи и коммуникации со студентами. По своей сути такие платформы хорошо подходят для сопровождения уже существующего учебного процесса. Однако задача бронирования ассистентов находится немного в другой плоскости. Здесь важно не только добавить пользователя на курс или выдать ему определенные права, но и провести отдельный административный процесс: выбрать подходящего студента, согласовать его участие, зафиксировать заявку, отследить ее статус и при необходимости передать информацию менеджеру. В стандартной логике LMS такие действия обычно не выделены как самостоятельный процесс, поэтому использовать подобные системы для рассматриваемой задачи без существенной доработки затруднительно.

Еще одним решением, близким к образовательной среде, является Gradescope⁷. Этот сервис в первую очередь ориентирован на проверку работ и организацию оценивания. Он может быть полезен преподавателям и ассистентам на этапе, когда ассистент уже назначен на курс и участвует в проверке заданий или анализе результатов студентов. Однако сам процесс поиска и закрепления ассистента за преподавателем в нем не решается. В Gradescope нет логики согласования бронирования между преподавателем, студентом и менеджером, а также нет встроенной поддержки административных вопросов, связанных с оформлением документов, выплатами и внутренними регламентами. Поэтому данный сервис можно рассматривать скорее как инструмент для последующей работы ассистента, но не как замену разрабатываемой системе. Следовательно, подобный сервис может рассматриваться только как инструмент для отдельного этапа учебного процесса, но не как аналог разрабатываемой системы в целом.

Еще один класс решений связан с автоматической поддержкой пользователей. Например, Zendesk AI agents⁸ предназначены для автоматизации ответов на обращения пользователей в различных каналах поддержки. Notion AI⁹ и корпоративный поиск Notion позво-

⁴LMS, или Learning Management System, - это система управления обучением, предназначенная для размещения учебных материалов, организации курсов, выдачи и проверки заданий, коммуникации между участниками образовательного процесса и отслеживания результатов обучения.

⁵Официальный сайт Moodle: <https://moodle.org>, дата обращения: 25.04.2026.

⁶Официальный сайт Canvas LMS: <https://www.instructure.com/canvas>, дата обращения: 25.04.2026.

⁷Официальный сайт Gradescope: <https://www.gradescope.com>, дата обращения: 25.04.2026.

⁸Документация Zendesk AI agents: <https://support.zendesk.com/hc/en-us/articles/6970583409690-About-AI-agents>, дата обращения: 25.04.2026.

⁹Официальная страница Notion Enterprise Search: <https://www.notion.com/product/enterprise-search>, дата обращения: 25.04.2026.

ляют находить ответы в рабочем пространстве и подключенных приложениях. Эти решения показывают, что автоматизация поиска информации и ответов на вопросы является востребованным направлением. Однако подобные системы обычно являются внешними платформами общего назначения. Они не встроены в предметную модель бронирования ассистентов и не управляют самими заявками, ролями и статусами. Кроме того, использование внешних сервисов может быть нежелательным при работе с внутренними нормативными документами, если требуется локальное хранение данных и выполнение вычислений на стороне проекта.

Таким образом, существующие решения можно разделить на несколько групп. Табличные инструменты обеспечивают простое хранение данных, но плохо подходят для формализации сложного ролевого процесса. Сервисы бронирования и календарного планирования автоматизируют выбор времени, но не учитывают специфику образовательного ассистирования. LMS-системы хорошо поддерживают учебный процесс, но не решают административную задачу бронирования ассистентов. Системы автоматической поддержки пользователей позволяют отвечать на вопросы, но не являются частью специализированного процесса управления заявками. Поэтому ни одно из рассмотренных решений не покрывает одновременно все ключевые требования данной работы.

Разрабатываемый сервис отличается от перечисленных аналогов тем, что объединяет несколько функций в рамках единой предметной системы. С одной стороны, он предоставляет веб-интерфейс для преподавателей, студентов и менеджеров, позволяющий управлять бронированием ассистентов. С другой стороны, он включает RAG-помощника, который использует внутренние документы для ответа на административные вопросы. Кроме того, использование собственной серверной части, базы данных PostgreSQL¹⁰, так как данная система управления базами данных хорошо подходит для работы со связанными сущностями, такими как пользователи, заявки, роли и бронирования и локальной языковой модели позволяет контролировать структуру данных, логику обработки заявок и вычисления, связанные с автоматической поддержкой пользователей.

В следующих разделах данной главы будут рассмотрены основные принципы построения разрабатываемого сервиса, особенности ролевой модели, архитектурные решения и подход к интеграции RAG-помощника. Также будет показано, каким образом выбранная архитектура позволяет устранить ограничения существующих решений и адаптировать систему под конкретные процессы образовательной организации.

¹⁰PostgreSQL - это объектно-реляционная система управления базами данных с открытым исходным кодом. Официальная документация PostgreSQL: <https://www.postgresql.org/docs/current/intro-what-is.html>, дата обращения: 25.04.2026.

2.3 Обоснование выбора разрабатываемого подхода

После рассмотрения существующих решений можно сделать вывод, что ни один из готовых инструментов не закрывает задачу полностью. Табличные инструменты позволяют быстро хранить данные, но не задают строгую логику процесса. LMS-системы хорошо подходят для сопровождения учебных курсов, но не предназначены для отдельного административного сценария бронирования ассистентов. Сервисы календарного планирования помогают согласовывать время, однако не учитывают специфику образовательной организации, статусы заявок, роли пользователей и связанные с этим документы. Системы поддержки пользователей, в свою очередь, могут отвечать на вопросы, но не управляют самим процессом бронирования.

Именно поэтому для проекта не был выбран готовый универсальный инструмент. В данном случае требовался сервис, который повторяет реальный порядок работы с ассистентами, а не заставляет участников процесса подстраиваться под чужую логику. Да, собственная разработка занимает больше времени: нужно отдельно продумывать интерфейс, права доступа, хранение данных и обработку заявок. Но зато появляется возможность сразу заложить в систему те сущности, которые действительно используются в процессе: преподавателей, студентов, менеджеров, учебные группы, заявки, статусы и связанные с ними документы.

Для этой задачи узкая специализация является скорее преимуществом. Универсальная платформа обычно предлагает широкий набор возможностей, но часть из них остается лишней, а нужные действия приходится настраивать вручную или обходными способами. В разрабатываемом сервисе логика изначально строится вокруг бронирования ассистентов. Поэтому можно заранее определить, кто создает заявку, кто выбирает ассистента, где отображается статус группы и в какой момент пользователю может понадобиться помощь по регламентам или инструкциям.

Выбор технологий также был связан не только с их популярностью. Важно было собрать такой стек, который можно поддерживать в рамках одного проекта и постепенно расширять без полной переработки архитектуры. Кроме того, при самостоятельной разработке особенно важно, чтобы по выбранным инструментам было достаточно учебных материалов, примеров и обсуждений типовых ошибок. Это снижает риск застрять на настройке окружения или на базовой инфраструктуре вместо работы над логикой сервиса.

По этой причине в основу была положена клиент-серверная архитектура. Она достаточно понятна для реализации и хорошо подходит для системы, где есть веб-интерфейс, отдельная серверная логика, база данных и дополнительный модуль RAG-помощника. Такой

вариант позволяет отдельно развивать визуальную часть, backend-логику и интеллектуальный сервис, не смешивая все компоненты в одном месте.

Клиентская часть реализуется с использованием **React**¹¹. Данный инструмент был выбран потому, что интерфейс системы состоит из большого количества повторяющихся элементов: карточек заявок, списков студентов, форм, кнопок действий, таблиц и диалоговых окон. Использование шаблонизированного подхода позволяет не писать каждый элемент заново, а выделять общие части интерфейса и переиспользовать их в разных разделах приложения. Это особенно важно при наличии нескольких ролей, так как преподаватель, студент и менеджер могут видеть разные страницы, но многие элементы интерфейса остаются похожими по структуре.

Для стилизации интерфейса используется **Tailwind CSS**¹². Выбор данного инструмента связан с тем, что он позволяет быстро настраивать внешний вид элементов прямо в разметке, не создавая большое количество отдельных CSS-файлов. Для проекта, в котором важна доработка визуальной части и удобство пользовательского интерфейса, это оказалось практичным решением. Кроме того, такой подход упрощает поддержку единого визуального стиля: одинаковые отступы, размеры, цвета и состояния элементов можно задавать более последовательно.

Основная серверная часть веб-приложения построена на **Node.js**¹³ и **Fastify**¹⁴. Этот выбор связан с тем, что данные технологии хорошо подходят для реализации REST API, обработки запросов от клиентской части и организации бизнес-логики. Сервер отвечает за регистрацию и авторизацию пользователей, работу с ролями, получение и изменение данных о заявках, бронированиях и связанных сущностях.

Отдельного внимания требует модуль RAG-помощника. Для его реализации был выбран язык **Python**¹⁵. Это связано с тем, что именно в **Python** собрано большое количество библиотек для обработки текстов, работы с языковыми моделями, эмбедингами и поиском

¹¹**React** - JavaScript-библиотека для разработки пользовательских интерфейсов на основе компонентного подхода. Официальная документация React: <https://react.dev/>, дата обращения: 25.04.2026.

¹²**Tailwind CSS** - CSS-фреймворк, основанный на использовании утилитарных классов для быстрой настройки внешнего вида элементов интерфейса. Официальная документация Tailwind CSS: <https://tailwindcss.com/docs>, дата обращения: 25.04.2026.

¹³**Node.js** - среда выполнения JavaScript вне браузера, которая может использоваться для разработки серверных приложений. Официальная документация Node.js: <https://nodejs.org/api/documentation.html>, дата обращения: 25.04.2026.

¹⁴**Fastify** 5 - минималистичный веб-фреймворк для **Node.js**, предназначенный для создания веб-приложений и API. Официальная документация Fastify 5: <https://expressjs.com/>, дата обращения: 25.04.2026.

¹⁵**Python** - высокоуровневый язык программирования общего назначения, широко используемый для разработки серверных приложений, обработки данных и работы с моделями машинного обучения. Официальная документация Python: <https://docs.python.org/3/>, дата обращения: 25.04.2026.

по документам. В отличие от основной серверной части, RAG-модуль выполняет не только обработку обычных API-запросов, но и более специфические операции: подготовку документов, разбиение текста на фрагменты, построение поискового индекса, извлечение релевантного контекста и формирование ответа с использованием локальной языковой модели.

Разделение основного сервера и RAG-модуля на разные части является осознанным архитектурным решением. Основное приложение на `Node.js` отвечает за пользовательские сценарии, работу с интерфейсом и базой данных, а сервер на `Python` отвечает за интеллектуальную обработку запросов, что позволяет сделать систему более гибкой. Например, при необходимости можно дорабатывать RAG-помощника, менять модель или способ поиска по документам, не переписывая при этом основную логику сервиса бронирования.

Использование локальной языковой модели также связано со спецификой предметной области. Внутренние положения, инструкции, шаблоны договоров и документы, связанные с оплатами, могут содержать персональную информацию, которую нежелательно передавать во внешние сервисы. Локальный запуск модели позволяет лучше контролировать обработку данных и снижает зависимость проекта от сторонних API. При этом такой подход имеет и свои ограничения: локальная модель может требовать дополнительных вычислительных ресурсов, а качество ее ответов необходимо проверять отдельно. Однако для рассматриваемой задачи преимущество в виде контроля над данными является достаточно существенным.

Для развертывания системы используются `Docker`¹⁶ и `Docker Compose`¹⁷. Это решение было выбрано для того, чтобы упростить запуск проекта и сделать окружение более воспроизводимым.

Итоговая версия сервиса будет размещена на виртуальной машине, на которой будут настроены необходимые открытые порты для работы клиентской и серверной частей приложения. Доступ пользователей к системе будет осуществляться по протоколу `HTTPS`, что позволит обеспечить защищенную передачу данных между браузером пользователя и сервером.

В качестве базы данных используется `PostgreSQL`. Данный выбор обусловлен тем, что в системе присутствует большое количество связанных сущностей: пользователи, роли, заявки, бронирования, статусы, документы и история изменений. Реляционная база данных хорошо подходит для такого типа информации, поскольку позволяет явно описывать связи

¹⁶`Docker` - платформа для разработки, доставки и запуска приложений в контейнерах. Официальная документация `Docker`: <https://docs.docker.com/>, дата обращения: 25.04.2026.

¹⁷`Docker Compose` - инструмент для описания и запуска многоконтейнерных приложений с помощью единого конфигурационного файла. Официальная документация `Docker Compose`: <https://docs.docker.com/compose/>, дата обращения: 25.04.2026.

между таблицами и контролировать целостность данных. Для рассматриваемой задачи это важно, так как ошибка в данных может привести не только к некорректному отображению интерфейса, но и к неправильной обработке административного процесса.

Таким образом, выбранный технический подход связан не только с удобством разработки, но и с особенностями самой задачи. Разрабатываемый сервис должен одновременно поддерживать понятный пользовательский интерфейс, ролевую модель, надежное хранение связанных данных, отдельную административную логику и модуль интеллектуальной поддержки. Комбинация `React`, `Tailwind CSS`, `Node.js`, `Fastify 5`, `PostgreSQL`, `Python`, локальной языковой модели и контейнеризации позволяет разделить эти задачи между разными компонентами и при этом сохранить систему достаточно простой для дальнейшего развития.

2.4 Выводы по первой главе

В первой главе была рассмотрена предметная область, связанная с организацией бронирования ассистентов преподавателей. Было показано, что данный процесс нельзя рассматривать только как простую запись студента в таблицу или назначение его на курс. На практике он включает несколько участников, различные права доступа, согласование заявок, контроль статусов, работу с документами и необходимость учитывать внутренние правила образовательной организации.

В первой главе также были разобраны инструменты, которые на первый взгляд могли бы заменить отдельную разработку. Часть из них действительно может быть полезна в похожих задачах. Например, таблицы подходят для быстрого сбора данных, LMS-системы хорошо работают внутри учебных курсов, а сервисы бронирования помогают согласовывать расписание. Однако в рассматриваемом процессе все эти возможности оказываются разрозненными. Ни один из таких инструментов сам по себе не описывает полный путь от создания заявки на курс до выбора ассистента и дальнейшего сопровождения этого назначения.

Из этого следует, что для данной задачи нужен не просто еще один способ хранить информацию, а отдельный сервис, построенный вокруг самого процесса ассистирования. В нем должны быть заранее заложены роли менеджера, преподавателя и студента, а также понятная логика работы с курсами, группами и назначениями. Тогда пользователю не нужно разбираться в структуре общей таблицы или искать нужную переписку: необходимые действия сразу находятся в интерфейсе, который соответствует его роли.

Кроме того, в ходе анализа стало понятно, что значительная часть трудностей связана не только с выбором ассистента, но и с вопросами по правилам. Пользователям приходится

обращаться к положениям, инструкциям, шаблонам договоров и правилам оплаты. Поэтому в проект был добавлен RAG-помощник. Его задача состоит в том, чтобы находить ответ не в общем виде, а с опорой на документы, которые используются внутри образовательной организации. Такой подход делает справочную поддержку более полезной, особенно в ситуациях, где важно соблюдать действующие регламенты.

Кроме того, в главе был обоснован выбор технических инструментов, используемых при разработке. Клиентская часть на `React` и `Tailwind CSS` позволяет реализовать удобный интерфейс для разных ролей пользователей. Серверная часть на `Node.js` и `Fastify 5` отвечает за обработку основных пользовательских сценариев и бизнес-логику. Для хранения связанных данных используется `PostgreSQL`. RAG-модуль вынесен в отдельный сервер на `Python`, так как именно эта экосистема лучше подходит для работы с текстами, языковыми моделями и поиском по документам. Использование `Docker` и `Docker Compose` позволяет упростить развертывание системы и сделать запуск проекта более воспроизводимым.

Таким образом, по результатам первой главы можно сделать вывод, что разрабатываемый сервис занимает промежуточное положение между системой бронирования, административной панелью и инструментом автоматической поддержки пользователей. Именно объединение этих функций в одном решении отличает проект от рассмотренных аналогов и определяет дальнейшую постановку задачи.

3 Постановка задачи и требования к системе

После анализа предметной области и существующих решений необходимо перейти к более точному и формальному описанию того, какую именно задачу должна решать разрабатываемая система. В первой главе было показано, что процесс бронирования ассистентов нельзя полностью перенести в готовые универсальные сервисы без потери части логики. Поэтому в рамках данной работы разрабатывается отдельный программный продукт, ориентированный на конкретный административный процесс образовательной организации.

Основная идея системы заключается в том, чтобы заменить разрозненную работу с таблицами, переписками и отдельными инструкциями единым веб-сервисом. Пользователь должен иметь возможность войти в систему, увидеть доступные для своей роли действия, выполнить необходимые операции с бронированиями и при необходимости получить справочную информацию через встроенного RAG-помощника. При этом система должна быть понятной не только с технической точки зрения, но и с точки зрения реального пользователя, который не обязан понимать внутреннюю структуру базы данных или логику работы серверной части.

Разрабатываемая система должна поддерживать три основные роли: преподавателя, студента и менеджера. Такое разделение выбрано не случайно, так как именно эти участники чаще всего взаимодействуют в процессе бронирования ассистентов. Введение отдельной роли менеджера является важным отличием новой версии сервиса, поскольку в предыдущем варианте значительная часть административных действий оставалась вне системы. Менеджер создает заявки и контролирует корректность процесса, при необходимости помогая решать административные вопросы, преподаватель просматривает назначенные на него заявки и выбирает себе ассистента, студент выступая в качестве потенциального ассистента, может самостоятельно подавать заявку на желаемые группы.

Как уже упоминалось ранее, отдельной частью постановки задачи является внедрение RAG-помощника. Его внедрение связано с тем, что пользователи часто задают вопросы, ответы на которые находятся во внутренних документах. При этом поиск нужной информации вручную может быть неудобным, особенно если документы имеют разную структуру и относятся к разным аспектам процесса. Поэтому система должна не только хранить и обрабатывать данные о бронированиях, но и помогать пользователям ориентироваться в положениях, инструкциях, регламентах, шаблонах договоров и FAQ.

3.1 Цель и задачи разработки

Целью разработки является создание комплексного веб-сервиса для управления бронированием ассистентов и автоматической поддержки пользователей с использованием RAG-технологий.

Для достижения поставленной цели работа была разбита на несколько последовательных этапов.

1. Сначала необходимо было определить, какие группы пользователей будут работать с сервисом, чем отличаются их задачи и какие действия должны быть доступны каждой роли.

2. Далее требовалось спроектировать структуру хранения данных. В нее должны входить сведения о пользователях, курсах, учебных группах, заявках, назначенных ассистентах и других сущностях, которые участвуют в процессе бронирования.

3. После этого необходимо было реализовать клиентскую часть приложения. Она должна предоставлять преподавателям, студентам и менеджерам понятный веб-интерфейс, через который можно выполнять основные действия без обращения к внешним таблицам.

4. Отдельной задачей стала разработка серверной части. Она отвечает за обработку запросов от интерфейса, проверку прав пользователей, выполнение основной логики сервиса и обмен данными с базой данных.

5. Важным этапом работы стало добавление роли менеджера. Эта роль нужна для того, чтобы создание заявок на курсы, контроль групп без ассистентов и отслеживание распределения нагрузки выполнялись внутри системы, а не вручную в отдельных файлах.

6. Также необходимо было реализовать RAG-помощника, который позволяет пользователям задавать вопросы по внутренним документам образовательной организации и получать ответы с опорой на найденные фрагменты этих документов.

7. Для хранения данных требовалось настроить использование PostgreSQL и организовать взаимодействие между основным backend-сервером, базой данных и отдельным RAG-сервисом.

8. Завершающим техническим этапом стала подготовка проекта к развертыванию. Для этого необходимо было описать запуск компонентов через Docker и Docker Compose, а также предусмотреть размещение сервиса на виртуальной машине с доступом по протоколу HTTPS.

3.2 Описание ролей пользователей

В разрабатываемой системе выделяются три основные роли: менеджер, преподаватель и студент, который в дальнейшем может стать ассистентом. Такое разделение необходимо для того, чтобы процесс бронирования был понятен для всех участников и при этом не смешивал между собой административные, преподавательские и студенческие действия. Каждый пользователь работает только с той частью системы, которая относится к его задачам.

Менеджер является пользователем, с которого начинается основной процесс работы с курсом. Он создает заявку на ведение курса и указывает в ней ключевую информацию: дисциплину, ответственного преподавателя, факультет, образовательную программу, количество групп, учебные модули и полезные ссылки. После этого менеджер может отслеживать, как распределяются ассистенты по группам, какие группы еще остаются без ассистентов и какая нагрузка уже закреплена за каждым студентом. Таким образом, роль менеджера связана с общей организацией и контролем процесса.

Преподаватель работает с теми курсами и группами, за которые он назначен ответственным. Для него система должна показывать текущее состояние групп: отдельно те, где ассистент еще не выбран, и отдельно те, где ассистент уже назначен. Основная задача преподавателя в системе заключается в выборе подходящего ассистента из числа зарегистрированных студентов. Такой подход позволяет преподавателю видеть актуальную ситуацию по своим группам без необходимости обращаться к отдельным таблицам или уточнять данные у менеджера.

Студент рассматривается в системе как потенциальный ассистент. Для него важно видеть, в какие группы он уже был выбран, а также какие группы еще доступны для ассистирования. Поэтому в интерфейсе студента предусмотрены два основных направления работы: просмотр собственных групп и поиск свободных групп. В первом случае студент получает информацию о уже назначенных ему курсах, во втором — может самостоятельно найти подходящую группу и отправить заявку на ассистирование.

Для всех трех ролей также доступен раздел с RAG-помощником. Он используется как общий справочный инструмент, к которому можно обратиться при возникновении вопросов о правилах, документах, оплате, сроках или других внутренних процедурах. При этом смысл обращения к помощнику может отличаться в зависимости от роли: менеджеру он помогает быстрее находить нужные пункты регламентов, преподавателю — уточнять порядок бронирования, а студенту — разбираться в условиях участия в ассистировании.

3.3 Функциональные требования

Функциональные требования описывают основные действия, которые должны быть доступны пользователям в системе. В рамках данной работы они связаны с тремя основными направлениями: созданием и сопровождением заявок на ведение курсов, выбором ассистентов для учебных групп и работой студентов с доступными возможностями ассистирования. Отдельным функциональным блоком является RAG-помощник, который должен быть доступен всем ролям.

К основным функциональным требованиям для роли менеджера относятся:

1. Система должна позволять менеджеру создавать заявку на ведение курса.
2. При создании заявки менеджер должен иметь возможность указать дисциплину, например анализ данных, программирование на Python, цифровую грамотность или машинное обучение.
3. Система должна позволять указывать ответственного преподавателя, факультет и образовательную программу.
4. Система должна поддерживать указание количества учебных групп в рамках заявки.
5. Система должна позволять выбирать учебные модули, в которых будет проходить курс, от первого до четвертого.
6. Система должна позволять добавлять к заявке полезные ссылки, например ссылку на курс в LMS, беседу в мессенджере, папку с материалами или другие ресурсы.
7. После создания заявки менеджер должен иметь возможность видеть группы, в которых ассистент еще не назначен.
8. Менеджер должен иметь возможность видеть группы, в которых ассистент уже выбран.
9. Система должна показывать менеджеру, сколько групп закреплено за каждым ассистентом.
10. Менеджер должен иметь возможность использовать RAG-помощника для поиска информации во внутренних документах.

К основным функциональным требованиям для роли преподавателя относятся:

1. Система должна показывать преподавателю группы, связанные с его курсами.
2. На странице преподавателя группы должны быть разделены на два блока: группы без ассистента и группы, где ассистент уже выбран.
3. Для групп без ассистента преподаватель должен иметь возможность просматривать

студентов, зарегистрированных в сервисе.

4. Преподаватель должен иметь возможность выбрать ассистента для конкретной учебной группы из числа зарегистрированных студентов.

5. После выбора ассистента система должна обновлять состояние группы и переносить ее в блок групп с выбранным ассистентом.

6. Преподаватель должен иметь возможность видеть информацию о группах, где ассистент уже назначен.

7. Преподаватель должен иметь возможность использовать RAG-помощника для уточнения правил бронирования, оформления документов и других связанных вопросов.

К основным функциональным требованиям для роли студента относятся:

1. Система должна предоставлять студенту раздел “мои группы”.

2. В разделе “мои группы” студент должен видеть группы, в которые он уже выбран ассистентом.

3. Система должна показывать студенту информацию о дисциплине, преподавателе, факультете, образовательной программе, учебных модулях и полезных ссылках по его группам.

4. Система должна предоставлять студенту раздел поиска.

5. В разделе поиска студент должен иметь возможность видеть группы, в которых ассистент еще не назначен.

6. Студент должен иметь возможность отправить заявку на ассистирование по выбранной группе.

7. Система должна сохранять информацию об отправленной заявке студента.

8. Студент должен иметь возможность использовать RAG-помощника для получения информации о документах, оплате, правилах участия и внутренних регламентах.

К общим функциональным требованиям можно отнести те возможности, которые не привязаны только к одной роли, а нужны для работы всей системы в целом.

1. В системе должна быть предусмотрена возможность добавления пользователей и указания их роли: менеджер, преподаватель или студент.

2. После входа пользователя сервис должен определять, какие разделы и действия ему доступны. Например, студент не должен видеть функции менеджера, а преподаватель не должен выполнять административные действия, не относящиеся к его курсам.

3. Система должна хранить основные данные, необходимые для процесса бронирования: пользователей, дисциплины, учебные группы, заявки на курсы, назначенных ассистентов и полезные ссылки.

4. Данные внутри системы должны быть связаны между собой. Заявка менеджера должна быть связана с ответственным преподавателем, образовательной программой, группами и студентами, которые были выбраны ассистентами.

5. После действий пользователя информация должна обновляться для остальных участников процесса. Например, если преподаватель выбрал ассистента для группы, менеджер должен видеть, что эта группа больше не остается свободной.

6. Для всех ролей должен быть доступен отдельный раздел с RAG-помощником.

7. Пользователь должен иметь возможность задать вопрос помощнику в свободной форме, без необходимости выбирать конкретный документ или раздел инструкции.

8. RAG-помощник должен искать подходящие фрагменты во внутренних документах и формировать ответ с учетом найденной информации.

В результате функциональные требования описывают не только отдельные кнопки или страницы, а общий порядок работы сервиса. Менеджер создает основу процесса через заявку на курс, преподаватель закрывает группы ассистентами, а студент может видеть свои назначения и искать новые возможности для участия. RAG-помощник дополняет эти сценарии: он не заменяет основные действия в системе, но помогает быстрее разобраться с правилами, документами и организационными вопросами.

3.4 Нефункциональные требования

Помимо конкретных функций, для сервиса важны и общие свойства, от которых зависит удобство его использования. Даже если все основные действия реализованы, система может оказаться неудобной, если интерфейс непонятен, данные хранятся неструктурированно или запуск проекта требует большого количества ручных настроек.

К основным нефункциональным требованиям относятся:

1. **Удобство использования.** Интерфейс должен быть достаточно простым, чтобы им могли пользоваться преподаватели, студенты и менеджеры без специальной технической подготовки. Пользователь должен быстро понимать, где находится нужный раздел и какое действие от него ожидается.

2. **Разграничение доступа.** Возможности пользователя должны зависеть от его роли. Менеджеру нужны инструменты контроля и создания заявок, преподавателю — выбор ассистентов для своих групп, студенту — просмотр назначений и поиск свободных групп. Смешивание этих действий сделало бы интерфейс менее понятным и могло бы привести к ошибкам.

3. Структурированное хранение данных. Информация о курсах, группах, пользователях и назначениях должна храниться в базе данных, а не в виде вручную редактируемых таблиц. Это позволяет лучше контролировать связи между объектами и уменьшает вероятность случайного изменения или потери данных.

4. Возможность развития системы. Проект не должен быть устроен так, что добавление новой функции требует переписывания значительной части кода. В дальнейшем может понадобиться добавить новые статусы, дополнительные поля в заявках, новые типы ссылок или расширенные возможности для менеджера.

5. Воспроизводимый запуск. Все компоненты проекта должны запускаться в заранее описанном окружении. Это особенно важно, потому что система состоит из frontend-части, backend-сервера, базы данных и отдельного RAG-модуля. Использование `Docker` и `Docker Compose` позволяет снизить количество ошибок при переносе проекта на другую машину.

6. Контроль обработки данных. Сервис работает с внутренними документами и административной информацией, поэтому важно не передавать такие данные во внешние сервисы без необходимости. По этой причине RAG-модуль ориентирован на работу с локальной языковой моделью.

7. Безопасность доступа. При размещении системы на виртуальной машине необходимо использовать защищенное соединение по протоколу `HTTPS`. Также должны быть явно настроены сетевые порты, чтобы наружу были доступны только те компоненты, которые действительно нужны пользователю.

Таким образом, нефункциональные требования задают условия, при которых сервис может использоваться не только как демонстрационный прототип, но и как основа для дальнейшего развития. Они связаны с удобством интерфейса, надежностью хранения данных, безопасностью, контролем доступа и возможностью развернуть систему в повторяемом окружении.

3.5 Требования к RAG-помощнику

RAG-помощник является отдельным компонентом системы, поэтому к нему предъявляется набор специальных требований. В отличие от обычного справочного раздела, помощник должен работать с вопросами, сформулированными пользователем в свободной форме. Это важно, так как преподаватель, студент или менеджер не всегда знает точное название документа или раздела, в котором находится нужная информация.

К основным требованиям к RAG-помощнику относятся:

1. Помощник должен принимать текстовый вопрос пользователя.
2. Помощник должен выполнять поиск релевантных фрагментов во внутренней базе документов.
3. Ответ должен формироваться с учетом найденного контекста, а не только на основе общих знаний языковой модели.
4. В качестве источников должны использоваться внутренние нормативно-правовые положения, инструкции по оплатам, регламенты бронирования, шаблоны договоров и FAQ.
5. RAG-модуль должен быть реализован отдельно от основного сервера веб-приложения.
6. Вычисления RAG-модуля должны выполняться на стороне проекта с использованием локальной языковой модели.
7. Система должна допускать обновление базы документов без необходимости полной переработки основного веб-приложения.
8. Ответы помощника должны быть ориентированы на практическую помощь пользователю, а не на абстрактное рассуждение.

Для данной работы важно, что RAG-помощник не рассматривается как полностью автономный заменитель менеджера. Он выполняет вспомогательную функцию и помогает пользователю быстрее найти нужную информацию. Если вопрос требует административного решения или проверки конкретной ситуации, окончательное действие все равно остается за ответственным сотрудником. Такой подход позволяет использовать преимущества языковых моделей, но при этом не переносить на них полный контроль над процессом.

3.6 Выводы по второй главе

Во второй главе задача была описана уже не в общем виде, а через требования к будущей системе. Здесь было важно зафиксировать, что сервис должен объединять несколько частей, которые раньше существовали отдельно: работу с заявками на курсы, выбор ассистентов для групп, хранение связанных данных и справочную поддержку пользователей через RAG-помощника.

Отдельно были рассмотрены роли пользователей. В системе выделяются менеджер, преподаватель и студент. Менеджер отвечает за создание заявки и контроль распределения ассистентов, преподаватель работает со своими группами и выбирает подходящих студентов, а студент видит уже назначенные ему группы и может искать новые возможности для ассистирования. Такое разделение помогает приблизить структуру сервиса к реальному процессу,

а не превращать его в общий список записей без понятной логики.

Кроме того, во второй главе были сформулированы функциональные и нефункциональные требования. Первые описывают основные действия в системе: создание заявок, работу с ролями, назначение ассистентов, обновление статусов и обращение к RAG-помощнику. Вторые задают более общие свойства сервиса: понятный интерфейс, разграничение доступа, надежное хранение данных, возможность дальнейшего расширения, воспроизводимый запуск и контроль обработки внутренней информации.

Отдельно были сформулированы требования к RAG-помощнику. Было отмечено, что данный модуль должен использовать внутренние документы образовательной организации и помогать пользователям находить ответы на административные вопросы. При этом помощник рассматривается как инструмент поддержки, а не как полная замена менеджера.

Таким образом, во второй главе была подготовлена основа для дальнейшего проектирования архитектуры системы. На следующем этапе необходимо описать, каким образом выделенные требования отражаются в структуре приложения, базе данных, серверной логике и пользовательском интерфейсе.

4 Проектирование архитектуры системы

4.1 Общая архитектура сервиса

После формулирования требований к системе необходимо перейти к описанию её архитектуры. В рамках данной работы архитектура рассматривается не только как техническая схема взаимодействия компонентов, но и как способ отразить реальные процессы, происходящие при бронировании ассистентов. Система должна быть понятна с точки зрения пользователя, удобна для дальнейшей разработки и достаточно гибка для расширения, так как в дальнейшем могут появиться новые роли, типы заявок, документы или дополнительные административные сценарии.

При проектировании архитектуры учитывалось, что разрабатываемый сервис объединяет несколько разных по смыслу частей. С одной стороны, это обычное веб-приложение, в котором пользователи работают с курсами, группами, заявками и ассистентами. С другой стороны, в систему добавляется RAG-помощник, который обрабатывает текстовые вопросы и обращается к внутренней базе документов. Поэтому архитектура сервиса была построена таким образом, чтобы основная логика бронирования и интеллектуальная обработка запросов не смешивались в одном компоненте. Такой подход соответствует общему принципу

разделения ответственности, который часто используется при проектировании программных систем [2].

Общая архитектура сервиса включает четыре основные части: клиентскую часть, основной сервер приложения, базу данных и отдельный сервер RAG-помощника. Клиентская часть отвечает за отображение интерфейса для трёх ролей пользователей: менеджера, преподавателя и студента. Основной сервер обрабатывает запросы от клиента, выполняет бизнес-логику, связанную с заявками и бронированиями, а также взаимодействует с базой данных. База данных используется для хранения информации о пользователях, ролях, курсах, группах, назначениях ассистентов и связанных сущностях. RAG-сервер отвечает за обработку вопросов пользователей, поиск релевантных фрагментов во внутренних документах и формирование ответа с использованием локальной языковой модели.

На уровне пользовательского взаимодействия система работает следующим образом. Менеджер создаёт заявку на ведение курса и указывает в ней дисциплину, ответственного преподавателя, факультет, образовательную программу, количество групп, учебные модули и полезные ссылки. После этого информация сохраняется в базе данных и становится доступна другим участникам процесса. Преподаватель, зайдя в систему, видит свои группы, разделённые на группы без ассистента и группы, где ассистент уже выбран. Студент, в свою очередь, может просматривать свои назначенные группы и искать группы, где ассистент ещё не назначен. Таким образом, один и тот же набор данных используется разными ролями, но отображается для них по-разному.

С точки зрения архитектуры это означает, что интерфейс не должен самостоятельно принимать решения о логике процесса. Он должен получать от сервера уже подготовленные данные, соответствующие роли пользователя и его правам. Например, преподаватель не должен видеть административные функции менеджера, а студент не должен получать доступ к действиям, связанным с созданием заявки на курс. Поэтому разграничение ролей реализуется не только на уровне отображения страниц, но и на уровне серверной логики. Такой подход позволяет избежать ситуации, когда ограничения существуют только в интерфейсе и могут быть обойдены при прямом обращении к API.

Для описания архитектуры программных систем часто используется представление через несколько точек зрения: логическую, процессную, физическую и сценарную. Подобная идея описана в модели «4+1» Кручтена, где архитектура рассматривается не как одна диаграмма, а как набор взаимосвязанных представлений, каждое из которых отражает интересы разных участников проекта [3]. В данной работе такой подход также является полезным. Логическое представление показывает основные сущности системы: пользовате-

лей, роли, курсы, группы, заявки и ассистентов. Процессное представление описывает обмен запросами между клиентом, сервером, базой данных и RAG-модулем. Физическое представление связано с развёртыванием компонентов в контейнерах и размещением на виртуальной машине. Сценарное представление отражает основные пользовательские действия: создание заявки менеджером, выбор ассистента преподавателем, подача заявки студентом и обращение к RAG-помощнику.

Отдельное внимание при проектировании было уделено RAG-модулю. Его было решено вынести в отдельный сервер, реализованный на языке `Python`. Это связано с тем, что обработка документов, построение эмбедингов, поиск релевантных фрагментов и запуск локальной языковой модели относятся к отдельному классу задач и имеют другую техническую природу по сравнению с обычной обработкой заявок. Основной сервер на `Node.js` и `Fastify` отвечает за работу веб-сервиса, а RAG-сервер получает от него текстовый вопрос, выполняет поиск по внутренним документам и возвращает готовый ответ. Такое разделение делает систему более понятной и позволяет изменять RAG-пайплайн независимо от основной логики бронирования.

Подобное разделение компонентов близко к сервисному подходу, при котором разные части системы имеют собственные зоны ответственности и взаимодействуют друг с другом через явно определённые интерфейсы. Ньюман отмечает, что при проектировании распределённых систем важно не просто разделить код на несколько сервисов, а определить границы ответственности каждого компонента [4]. В рассматриваемом проекте такими границами являются основная логика бронирования и логика интеллектуальной справочной поддержки. При этом система не является сложной микросервисной архитектурой в полном смысле этого слова, так как проект остаётся сравнительно небольшим. Однако выделение RAG-модуля в отдельный сервис позволяет не перегружать основной backend и делает дальнейшее развитие более удобным.

Взаимодействие между компонентами можно описать следующим образом. Пользователь работает с интерфейсом в браузере. Клиентская часть отправляет запросы на основной сервер. Если запрос связан с курсами, группами, пользователями или назначениями ассистентов, сервер обращается к базе данных `PostgreSQL`, получает или изменяет данные и возвращает результат клиенту. Если пользователь задаёт вопрос RAG-помощнику, основной сервер передаёт этот вопрос на `Python`-сервер RAG-модуля. После обработки вопроса RAG-сервер возвращает ответ, который затем отображается пользователю в интерфейсе. Общая схема взаимодействия компонентов представлена на рисунке 4.1.

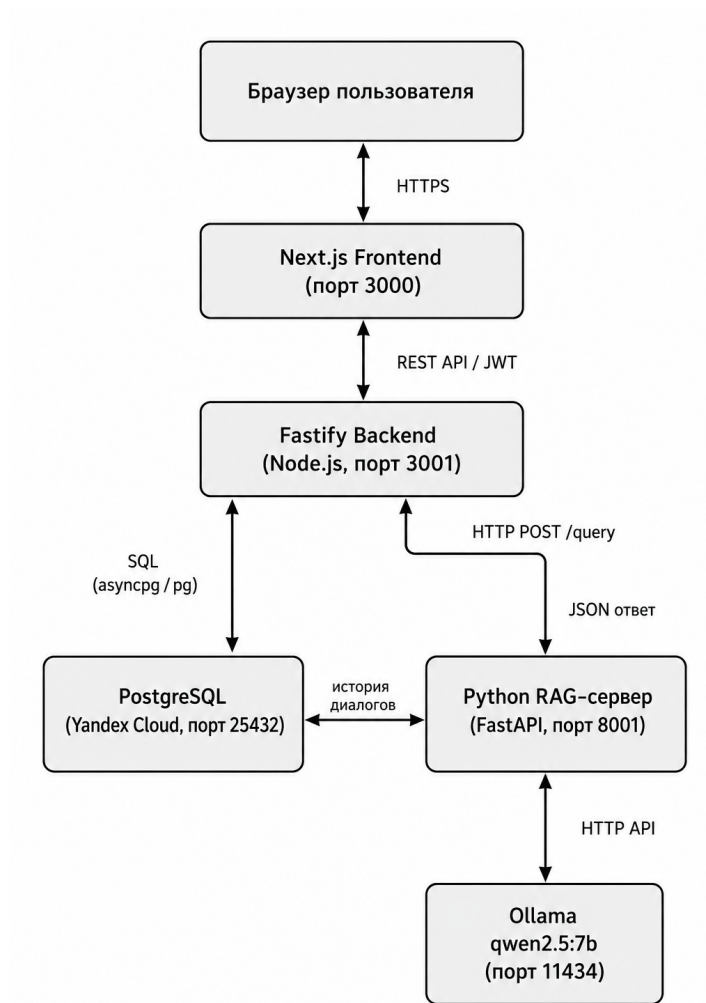


Рис. 4.1: Общая архитектура сервиса

Каждый компонент системы описан как отдельный контейнер: клиентская часть, основной сервер, база данных и RAG-сервер. Для их совместного запуска используется **Docker Compose**. Сервис размещается на виртуальной машине, где настроен доступ по протоколу **HTTPS** и открыты необходимые порты. Это позволяет пользователям работать с системой через браузер, а внутренние компоненты взаимодействуют между собой в рамках настроенного серверного окружения.

Таким образом, общая архитектура сервиса строится вокруг разделения пользовательского интерфейса, основной бизнес-логики, хранения данных и интеллектуальной поддержки. Такое решение позволяет сохранить систему достаточно простой, но при этом не смешивать разные по смыслу части проекта. В следующих подразделах каждая из этих частей будет рассмотрена более подробно.

4.2 Структура базы данных

База данных является центральным хранилищем состояния системы. В ней сосредоточена вся информация о пользователях, курсах, группах, заявках на бронирование и истории диалогов с RAG-помощником. Поскольку данные в системе тесно связаны между собой — студент подаёт заявку на группу, группа принадлежит курсу, курс привязан к дисциплине и образовательной программе — для хранения была выбрана реляционная база данных. Реляционная модель хорошо подходит для подобных задач, так как позволяет явно описать связи между сущностями и обеспечить целостность данных через механизм внешних ключей [5].

В качестве конкретной СУБД используется PostgreSQL. Этот выбор обусловлен несколькими соображениями. Во-первых, PostgreSQL поддерживает пользовательские типы данных — в частности, перечисления (ENUM), которые используются для описания ролей пользователей, статусов заявок и типов оплаты. Во-вторых, система обеспечивает транзакционную целостность (ACID), что важно при одновременном изменении связанных записей, например при назначении ассистента и обновлении статуса группы. В-третьих, PostgreSQL хорошо интегрируется с используемыми в проекте библиотеками: pg на стороне Node.js-сервера и asyncpg на стороне Python RAG-сервера. Все таблицы размещены в отдельной схеме `dc_new`, что позволяет логически отделить данные сервиса от других возможных таблиц в той же базе.

Центральной сущностью схемы является таблица `users`. Она хранит записи для всех трёх ролей системы: менеджера, преподавателя и студента. Роль пользователя описывается через перечисление `user_role` и определяет, какие маршруты и действия доступны конкретному пользователю. Пароли хранятся в виде хешей, рассчитанных с помощью `bcrypt`, а не в открытом виде. Для студентов предусмотрена отдельная таблица `student_profiles`, связанная с `users` через внешний ключ. В ней хранятся расширенные данные: год обучения, факультет, образовательная программа, контактные данные и достижения. Такое разделение позволяет не перегружать основную таблицу пользователей полями, актуальными только для одной роли.

Логика бронирования строится вокруг нескольких взаимосвязанных таблиц. Таблица `faculties` хранит список факультетов, `programs` — образовательные программы, привязанные к факультетам, `disciplines` — дисциплины, а `modules` — учебные модули с номерами от одного до пяти. На основе этих справочников менеджер формирует заявку на ведение курса, которая записывается в таблицу `course_offers`. Каждая заявка связана с конкретным преподавателем, менеджером, дисциплиной и образовательной программой. Статус за-

явки описывается перечислением `offer_status` и может принимать значения `open`, `closed` и `archived`. Связь между заявкой и учебными модулями реализована через промежуточную таблицу `course_offer_modules`, поскольку один курс может охватывать несколько модулей одновременно. Дополнительно к каждой заявке могут быть прикреплены ссылки на учебные материалы, которые хранятся в таблице `course_offer_links`.

Заявки студентов на ассистирование фиксируются в таблице `bookings`. Каждая запись связывает конкретного студента с конкретной заявкой на курс. В ней указывается тип оплаты — деньги или баллы (`payment_type`), количество групп, а также статус заявки: `pending`, `active` или `cancelled`. Если запись создаётся не самим студентом, а преподавателем, это фиксируется отдельным полем `created_by_teacher_id`. Таблица `student_preferences` дополняет эту логику: в ней студент может указать приоритет интереса к конкретным дисциплинам и желаемый размер группы, что может использоваться при автоматическом или ручном подборе ассистентов.

Отдельный блок таблиц обслуживает RAG-модуль. Таблица `chat_conversations` хранит сессии диалогов, привязанные к конкретному пользователю, а таблица `chat_messages` содержит отдельные сообщения внутри каждой сессии. Роль сообщения — пользователь или помощник — хранится в поле `role`, содержимое сообщения — в поле `content`. Такая структура позволяет RAG-серверу при каждом новом запросе загружать последние сообщения из базы и передавать их языковой модели в качестве контекста. Благодаря хранению истории в базе данных, а не в памяти сервиса, диалоги сохраняются между перезапусками контейнеров.

Для повышения производительности в схеме созданы индексы на внешних ключах и на поле `role` таблицы `users`. Это ускоряет наиболее частые операции: выборку групп по преподавателю, поиск заявок студента, фильтрацию пользователей по роли. Всего в схеме используется четыре перечисления: `user_role`, `offer_status`, `payment_type` и `booking_status`. Применение перечислений вместо произвольных строк исключает появление некорректных значений на уровне базы данных и упрощает проверку допустимых состояний на стороне сервера.

Полная структура схемы с указанием всех таблиц, столбцов и связей между ними представлена на рисунке [4.2](#).

Клиентская часть реализуется с использованием `Next.js 16` и `React 19`. Данный выбор связан с тем, что интерфейс сервиса состоит из набора повторяющихся элементов: карточек курсов, списков групп, форм создания заявок, кнопок действий, блоков статусов, таблиц и модальных окон. В `React` подобные элементы удобно выносить в отдельные компоненты, после чего переиспользовать их в разных частях приложения. Такой подход позволяет не дублировать разметку и делает интерфейс более удобным для дальнейшего изменения. Идея компонентной организации интерфейса является одной из базовых при разработке современных `React`-приложений [6].

Структурно клиентскую часть можно разделить на несколько уровней. Первый уровень составляют страницы, соответствующие основным разделам приложения. Для менеджера это страницы создания заявки на курс и просмотра распределения ассистентов по группам. Для преподавателя это страница со списком его групп, разделённая на группы без ассистента и группы с уже назначенным ассистентом. Для студента это раздел «мои группы» и раздел поиска свободных групп. Отдельным разделом интерфейса выступает RAG-помощник, доступный всем ролям.

Второй уровень составляют переиспользуемые компоненты. Например, карточка группы может использоваться в интерфейсе преподавателя, студента и менеджера, но с разными доступными действиями. В одном случае на карточке может отображаться кнопка выбора ассистента, в другом — кнопка подачи заявки, а в третьем — только административная информация. Такой подход позволяет сохранить единый визуальный стиль, но при этом не смешивать действия разных ролей. Кроме того, общими компонентами являются поля ввода, выпадающие списки, блоки с полезными ссылками, индикаторы статуса и элементы навигации.

Третий уровень связан с обменом данными между клиентом и сервером. Для этого в клиентской части выделяется отдельный слой функций, отвечающих за обращения к API. Страницы и визуальные компоненты не описывают детали HTTP-запросов напрямую. Вместо этого они обращаются к функциям получения списка групп, создания заявки или отправки вопроса RAG-помощнику. Такое разделение делает код более понятным: визуальная часть отвечает за отображение данных, а отдельный слой API — за получение и отправку информации на сервер.

Отдельное значение имеет маршрутизация внутри приложения. После входа в систему пользователь попадает в интерфейс, соответствующий его роли. Менеджер не видит страницы, предназначенные только для преподавателя, а студент не имеет доступа к действиям по созданию заявок на курс. Поэтому в клиентской части предусмотрено разделение маршрутов

по ролям. При этом важно понимать, что такое разделение используется прежде всего для удобства интерфейса: основная проверка прав всё равно выполняется на серверной стороне, так как ограничения только на уровне клиента не являются достаточной защитой.

Для визуальной части приложения был выбран Tailwind CSS. В данном проекте он оказался удобен прежде всего из-за скорости работы с интерфейсом. Многие элементы страниц нужно было часто менять в процессе разработки: где-то поправить отступы, где-то изменить расположение блоков, где-то сделать карточку или форму более читаемой. При использовании утилитарных классов такие изменения можно вносить прямо в разметке компонента, не переходя постоянно между разными файлами стилей.

Еще один плюс такого подхода связан с единообразием интерфейса. В системе есть несколько ролей, и для каждой из них предусмотрены свои страницы. При этом пользователь не должен ощущать, что он попадает в разные приложения. Карточки групп, кнопки, формы, блоки со статусами и навигация должны выглядеть похоже, даже если набор действий на страницах отличается. Поэтому единая система классов помогает быстрее поддерживать общий визуальный стиль и не разносить оформление по множеству несвязанных CSS-правил.

При проектировании клиентской части также учитывалось, что основное состояние системы должно находиться не в браузере, а на сервере. Пользовательский интерфейс получает данные через API и отображает их в зависимости от роли пользователя. Например, преподаватель видит свои группы и выбранных ассистентов, студент — свои назначения и доступные группы для поиска, а менеджер — общую картину по курсам и распределению ассистентов. После действия пользователя frontend отправляет запрос на backend и обновляет данные уже по ответу сервера. Такой вариант позволяет уменьшить риск расхождения данных между разными участниками процесса.

В итоге клиентская часть была спроектирована как набор страниц и компонентов, связанных с конкретными пользовательскими сценариями. Ее задача не сводится только к отображению данных из базы. Она должна сделать процесс бронирования понятным: показать пользователю нужный раздел, выделить доступные действия и не перегрузить интерфейс лишними элементами. Компонентный подход помогает переиспользовать одни и те же визуальные элементы, а разделение страниц по ролям делает работу с сервисом более предсказуемой.

4.4 Архитектура серверной части основного приложения

Серверная часть основного приложения отвечает за обработку пользовательских запросов, выполнение бизнес-логики и взаимодействие с базой данных. В рамках данного проекта сервер не является просто промежуточным слоем между интерфейсом и таблицами в базе данных. Он определяет, какие действия доступны пользователю, какие данные должны быть возвращены для конкретной роли и каким образом изменяется состояние системы после выполнения операции.

Основной сервер реализуется на `Node.js` с использованием фреймворка `Fastify` 5¹⁸ и языка `TypeScript`. Такая связка хорошо подходит для построения API, через который клиентская часть может получать и изменять данные. В рамках проекта сервер предоставляет набор маршрутов, связанных с пользователями, курсами, группами, заявками, назначением ассистентов и обращениями к RAG-помощнику. При этом серверная часть проектируется так, чтобы отдельные группы задач были разделены между разными слоями приложения.

В основе взаимодействия клиента и сервера лежит подход REST API. Пользовательские действия в интерфейсе преобразуются в HTTP-запросы к серверу: получение списка групп, создание заявки, выбор ассистента, отправка заявки студентом или обращение к RAG-помощнику. REST-подход удобен для такой системы, поскольку позволяет представить основные сущности приложения в виде ресурсов и работать с ними через стандартные методы HTTP. Впервые REST был подробно описан Филдингом как архитектурный стиль для сетевых приложений [7].

В серверной части можно выделить несколько логических уровней. Первый уровень — маршруты. Они определяют, по каким адресам клиентская часть может обращаться к серверу. Например, отдельные маршруты используются для получения курсов менеджера, списка групп преподавателя, доступных групп для студента или отправки вопроса RAG-помощнику. Второй уровень — контроллеры, которые принимают запрос, извлекают из него параметры и передают управление дальше. Третий уровень — сервисы, где находится основная бизнес-логика. Четвёртый уровень — работа с базой данных, где выполняются запросы к PostgreSQL.

Такое разделение нужно для того, чтобы серверный код не превращался в набор длинных обработчиков маршрутов. Например, если преподаватель выбирает ассистента для группы, сервер должен не только записать идентификатор студента в базу данных. Он также должен проверить, существует ли такая группа, относится ли она к курсу данного препода-

¹⁸Официальная документация Fastify: <https://fastify.dev/docs/latest/>, дата обращения: 18.05.2026.

вателя, не назначен ли уже ассистент, зарегистрирован ли выбранный студент в системе и имеет ли пользователь право выполнить это действие. Если вся эта логика находится прямо внутри маршрута, код быстро становится трудным для поддержки. Поэтому проверки и изменения состояния выносятся в сервисный слой.

Подобная организация соответствует распространённой практике разделения приложения на слои, где представление, бизнес-логика и доступ к данным не смешиваются между собой. Фаулер описывает такой подход как один из важных принципов построения корпоративных приложений, поскольку разделение ответственности упрощает сопровождение системы и изменение отдельных её частей [8]. В данной работе это особенно важно из-за наличия нескольких ролей и разных сценариев работы с одними и теми же данными.

Отдельная часть backend-логики связана с проверкой прав пользователя. После авторизации сервер получает не просто информацию о том, кто вошел в систему, но и роль этого пользователя. От нее зависит, какие действия вообще могут быть выполнены. Например, менеджер работает с созданием заявок на курсы, преподаватель выбирает ассистентов только для своих групп, а студент может просматривать доступные группы и подавать заявку на ассистирование. Такая проверка выполняется именно на стороне сервера. Это важно, потому что интерфейс можно изменить или попытаться отправить запрос напрямую, но без нужной роли сервер все равно не должен выполнять закрытое действие.

Кроме проверки прав, backend отвечает за то, чтобы данные в системе оставались согласованными. После того как менеджер создает заявку, в системе появляются связанные с ней группы. Эти группы затем становятся видны преподавателю и студентам в соответствующих разделах. Когда преподаватель выбирает ассистента, состояние группы меняется, и она больше не должна отображаться как свободная. Если студент отправляет заявку на ассистирование, эта информация также фиксируется в базе данных и может быть использована при дальнейшем выборе. За счет этого переходы между состояниями происходят не вручную, а через серверную логику.

Также в серверной части необходимо было предусмотреть обработку ошибок. Они могут появляться в разных ситуациях: пользователь заполнил форму некорректно, пытается выполнить действие без нужных прав, обращается к несуществующей группе, повторно назначает ассистента или отправляет вопрос в момент, когда RAG-сервер недоступен. В таких случаях backend должен вернуть не технический текст ошибки, а понятный ответ, который затем можно показать в интерфейсе. В Fastify для этого используются хуки и общий механизм обработки ошибок, позволяющий единообразно формировать ответы для разных маршрутов.

Логическая организация серверной части представлена на рисунке 4.3.

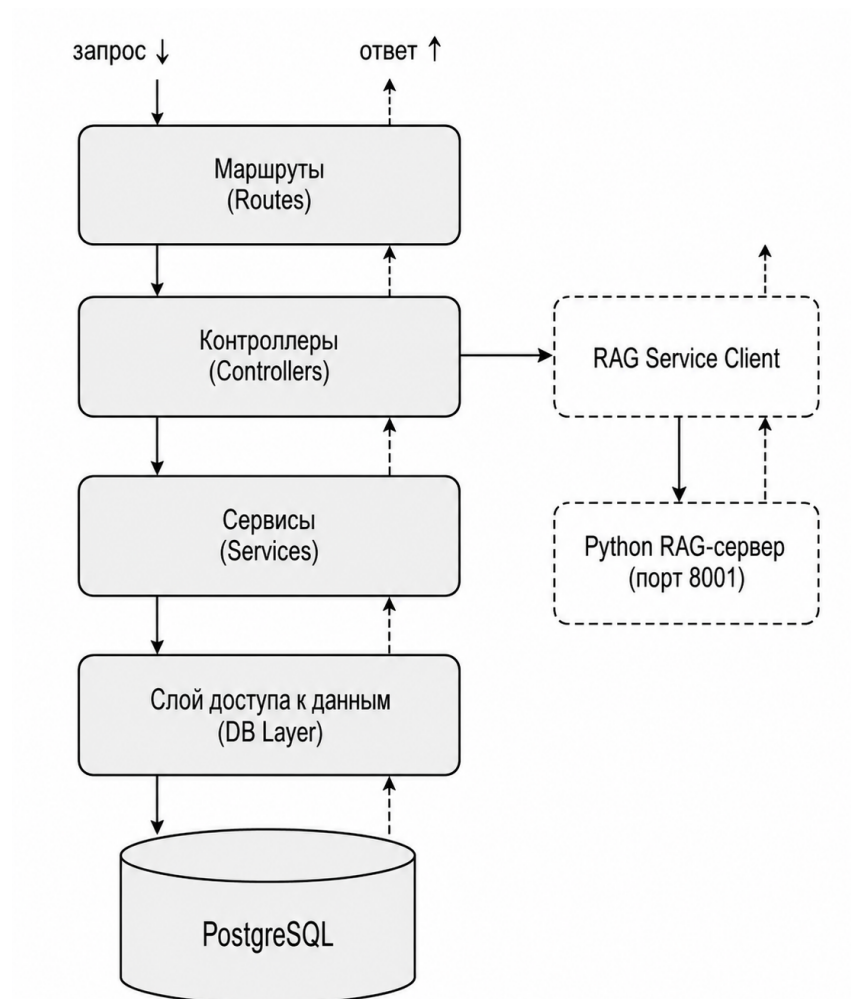


Рис. 4.3: Архитектура серверной части

Работа с RAG-помощником вынесена в отдельный внешний вызов. В этом случае контроллер принимает вопрос пользователя, передает его через клиент RAG-сервиса на Python-сервер, получает сгенерированный ответ и возвращает его обратно в интерфейс. Основной backend при этом не занимается поиском по документам и генерацией текста, а только связывает пользовательский запрос с отдельным RAG-модулем.

Таким образом, серверная часть выполняет роль центрального связующего слоя. Через нее проходят действия пользователей, проверки прав, изменения состояния заявок и групп, обращения к базе данных и запросы к RAG-сервису. Такое разделение помогает не смешивать интерфейсную логику, работу с данными и интеллектуальную обработку запросов в одном месте, а значит делает систему более понятной для дальнейшей поддержки и развития.

4.5 Архитектура RAG-модуля

Одной из ключевых особенностей разрабатываемого сервиса является встроенный интеллектуальный помощник, способный отвечать на вопросы пользователей на основе внутренних документов НИУ ВШЭ. Для его реализации был выбран подход RAG — Retrieval-Augmented Generation. Суть подхода состоит в том, что языковая модель не отвечает на вопрос самостоятельно, опираясь только на знания, накопленные в процессе обучения, а получает перед генерацией ответа набор релевантных фрагментов из заранее подготовленной базы документов. Это позволяет существенно снизить вероятность фактических ошибок при ответах на специфические вопросы предметной области [9]. Для системы, где пользователи задают вопросы о правилах проведения экзаменов, шкалах оценок и порядке сдачи долгов, такое ограничение области поиска ответов принципиально важно.

RAG-модуль реализован в виде отдельного Python-сервиса на базе FastAPI. Выбор Python обусловлен тем, что все основные библиотеки для работы с языковыми моделями и векторными представлениями — `sentence-transformers`, `numpy`, `httpx` — являются Python-нативными и не имеют полноценных аналогов в экосистеме Node.js. Вынесение RAG-функциональности в отдельный процесс позволяет избежать технологического смешения в основном сервере и изменять пайплайн независимо от логики бронирования.

База знаний

Исходными данными для модуля служит набор из 23 документов в формате Markdown, размещённых в директории `knowledge_base/`. В них собраны нормативные материалы НИУ ВШЭ: регламент проведения независимых экзаменов, спецификации заданий по программированию и анализу данных разных уровней сложности, процедуры апелляций, шкалы перевода оценок, даты экзаменов и часто задаваемые вопросы. Формат Markdown был выбран как удобный для хранения структурированного текста: заголовки второго и третьего уровня разграничивают смысловые разделы, что используется при разбиении документов на фрагменты.

Загрузка и чанкинг документов

Разбиение базы знаний на отдельные фрагменты выполняется в модуле `loader.py`. В качестве основной опоры используется структура самих Markdown-файлов. Документы уже содержат заголовки и подзаголовки, поэтому логично использовать их как естественные границы смысловых частей. Разделы, начинающиеся с заголовков второго и третьего

уровней, выделяются в отдельные блоки. Если такой блок получается слишком длинным, он дополнительно делится по пустым строкам, то есть фактически по абзацам.

После первичного разбиения короткие абзацы не оставляются отдельными фрагментами, а объединяются обратно, пока итоговый размер не приблизится к ограничению в 1200 символов. Такое ограничение выбрано не случайно. Используемая модель эмбедингов имеет ограничение на входную последовательность, а при работе с русским текстом и многоязычным токенизатором один токен не всегда соответствует одному слову. Поэтому размер фрагмента был выбран с запасом, чтобы не терять часть текста при кодировании. Для каждого фрагмента сохраняются не только его содержимое, но и метаданные: название документа, ближайший заголовок и имя исходного файла. Кроме того, название документа и заголовок добавляются в начало самого текста фрагмента. Это помогает модели учитывать не только отдельный абзац, но и его место внутри документа.

Векторный индекс

После подготовки фрагментов для каждого из них строится векторное представление. Для этого используется модель `paraphrase-multilingual-mpnet-base-v2` из библиотеки `sentence-transformers` [10]. Данная модель была выбрана из-за поддержки русского языка и возможности получать компактные семантические представления текста. Это важно, поскольку пользователь может формулировать вопрос не теми же словами, которые используются в документе. Например, в регламенте может быть написано «вознаграждение», а пользователь может спросить про «оплату». Обычный поиск по словам в такой ситуации может сработать хуже, чем поиск по смысловой близости.

На выходе модель формирует вектор размерности 768. Все такие векторы нормализуются по норме L2. После этого косинусное сходство между вопросом пользователя и фрагментом документа можно считать как скалярное произведение нормализованных векторов:

$$\text{sim}(q, c_i) = \hat{q} \cdot \hat{c}_i, \quad \hat{v} = \frac{v}{\|v\|} \quad (1)$$

Сам индекс хранится в оперативной памяти как матрица NumPy размера $N \times 768$, где N обозначает количество подготовленных фрагментов. В текущей версии проекта база знаний имеет небольшой объем и состоит из заранее определенного набора документов. Поэтому отдельное векторное хранилище не используется. Подключение таких решений, как FAISS или ChromaDB, имело бы смысл при большем объеме данных или при необходимости часто обновлять индекс, однако для текущего прототипа это усложнило бы архитектуру без

заметной практической пользы.

Пайплайн обработки запроса

Когда пользователь задает вопрос, основной backend передает его в RAG-сервис через эндпоинт `POST /query`. Далее обработка запроса проходит несколько этапов.

Сначала вопрос пользователя кодируется той же моделью эмбедингов, которая использовалась при построении индекса. Полученный вектор сравнивается со всеми векторами фрагментов из матрицы. После сортировки по значению сходства выбираются пять наиболее близких фрагментов. Параметр $k = 5$ вынесен в конфигурацию как `retrieval_top_k`. Такое значение было выбрано как компромисс: с одной стороны, модели нужно дать достаточно контекста, с другой — слишком большое количество фрагментов делает промпт длиннее и может ухудшить фокус ответа.

Затем RAG-сервис обращается к базе данных и получает историю текущего диалога. Для этого используется идентификатор `conversation_id`. Из истории берутся последние сообщения, но в промпт включается только ограниченное число ходов диалога. Это позволяет сохранить контекст разговора, но не перегружать модель старой информацией. Важным решением является то, что история хранится в PostgreSQL, а не в памяти Python-процесса. Благодаря этому диалоги не теряются при перезапуске контейнера RAG-сервера.

После этого формируется итоговый промпт для языковой модели. В него включаются системная инструкция, история диалога, найденные фрагменты документов и сам вопрос пользователя. В системной инструкции отдельно указано, что ответ должен строиться только на основе переданных документов. Если нужной информации во фрагментах нет, модель должна честно сообщить об этом, а не придумывать ответ. Для генерации используется локальная модель через API `Ollama`. Температура установлена равной 0,2, так как в данной задаче важнее стабильность и фактическая точность, чем разнообразие формулировок. Максимальная длина ответа ограничена 1024 токенами, а таймаут запроса составляет 120 секунд.

Выбор языковой модели

В качестве генеративной модели используется `qwen2.5:7b`, запускаемая локально через `Ollama`. Это открытая модель с семью миллиардами параметров из семейства Qwen2.5 [11]. Для проекта такой вариант оказался удобным по нескольким причинам. Во-первых, модель можно использовать без обращения к внешним API. Это важно, поскольку вопросы пользователей и внутренние документы образовательной организации не должны передаваться

сторонним сервисам без необходимости. Во-вторых, модель такого размера дает приемлемое качество ответов и при этом может запускаться локально в квантизованном виде, без отдельного серверного GPU.

Конфигурация модели не зашита в код напрямую. Название модели передается через переменную окружения `OLLAMA_MODEL`. Это оставляет возможность в дальнейшем заменить `qwen2.5:7b` на другую локальную модель, если потребуется улучшить качество ответов или уменьшить время генерации.

Общая схема пайплайна RAG-модуля представлена на рисунке 4.4.

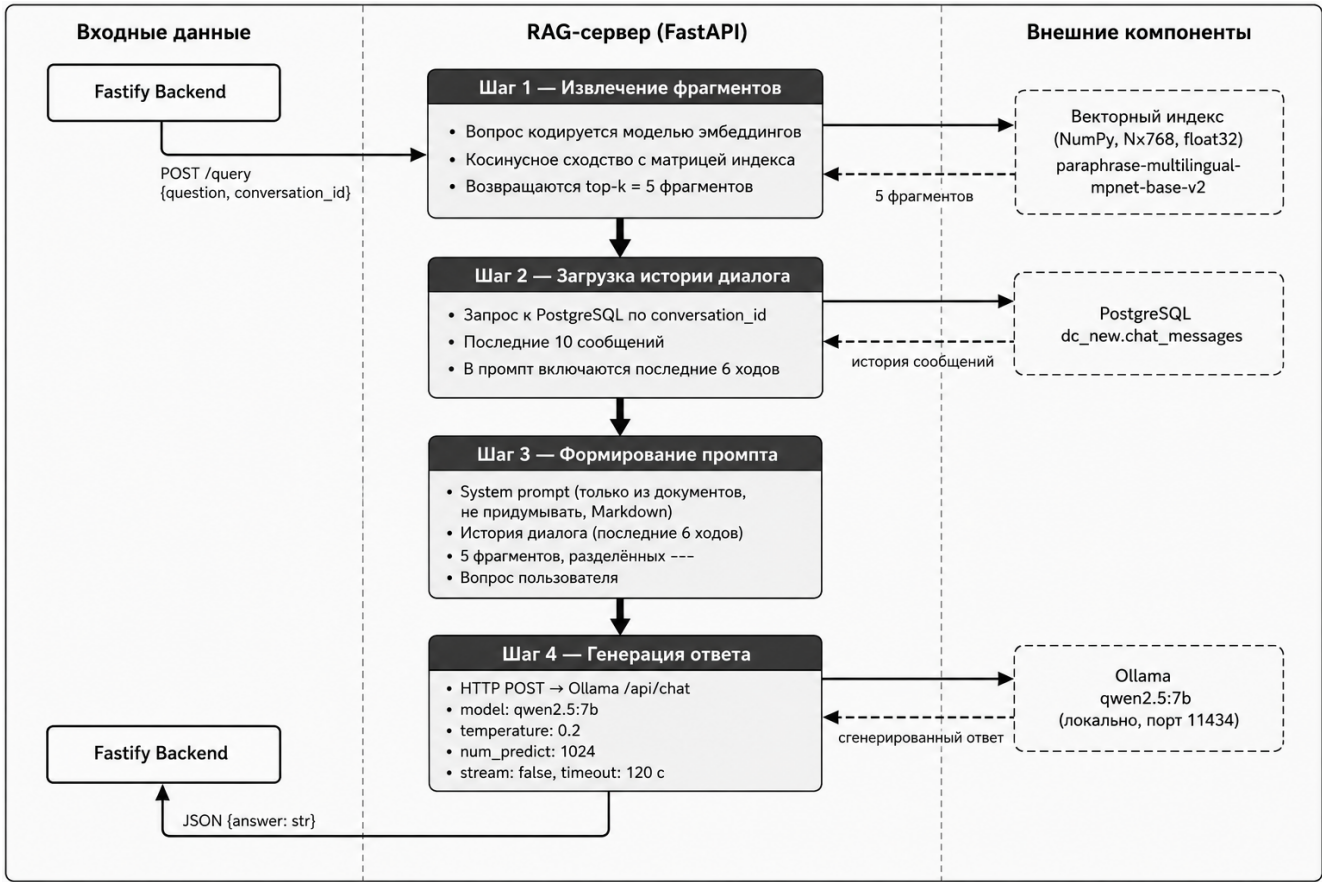


Рис. 4.4: Схема пайплайна RAG-модуля

Таким образом, RAG-модуль состоит из нескольких достаточно независимых частей. Загрузчик подготавливает документы и делит их на фрагменты, индекс отвечает за поиск близких по смыслу частей базы знаний, а генератор формирует итоговый ответ с помощью локальной языковой модели. История диалогов хранится в общей базе данных, поэтому RAG-сервис остается связанным с основным приложением, но при этом его внутренняя логика не смешивается с логикой бронирования ассистентов.

4.6 Развёртывание системы

Итоговая версия системы разворачивается с использованием контейнеризации на базе **Docker** и **Docker Compose**. Такой подход был выбран из-за того, что проект состоит из нескольких независимых компонентов, которые должны запускаться согласованно: frontend-приложения, основного backend-сервера, Python RAG-сервера и локального сервера языковой модели. Если запускать каждый компонент вручную, возрастает вероятность ошибок, связанных с версиями зависимостей, переменными окружения, портами и порядком старта сервисов. Использование контейнеров позволяет описать окружение каждого компонента отдельно, а **Docker Compose** — объединить их в единую конфигурацию запуска¹⁹.

В контейнеризированной схеме система включает четыре основных сервиса. Первый сервис — frontend на **Next.js** 16 и **React** 19, доступный на порту 3000. Он отвечает за пользовательский интерфейс и взаимодействие с браузером. Второй сервис — основной backend на **Node.js**, **Fastify** 5 и **TypeScript**, работающий на порту 3001. Он реализует REST API, обрабатывает авторизацию, пользовательские роли, заявки, курсы, группы и назначения ассистентов. Третий сервис — Python RAG-сервер на **FastAPI**, доступный на порту 8001. Он принимает вопросы пользователей от backend-сервера, выполняет поиск релевантных фрагментов в базе знаний и формирует запрос к локальной языковой модели. Четвёртый сервис — **Ollama**, который обслуживает локальную модель **qwen2.5:7b** на порту 11434.

База данных **PostgreSQL** 12+ размещается отдельно в **Yandex Cloud**. К ней обращаются два сервиса: основной backend и RAG-сервер. Backend использует базу данных для хранения пользователей, ролей, курсов, заявок, групп и назначений ассистентов. RAG-сервер использует ту же базу для хранения истории диалогов, что позволяет не терять пользовательские сессии при перезапуске контейнера. Такой вариант удобен для прототипа, поскольку основное состояние системы хранится вне контейнеров и не зависит от их пересоздания. Схема развёртывания представлена на рисунке 4.5.

¹⁹Официальная документация **Docker Compose**: <https://docs.docker.com/compose/>, дата обращения: 18.05.2026.

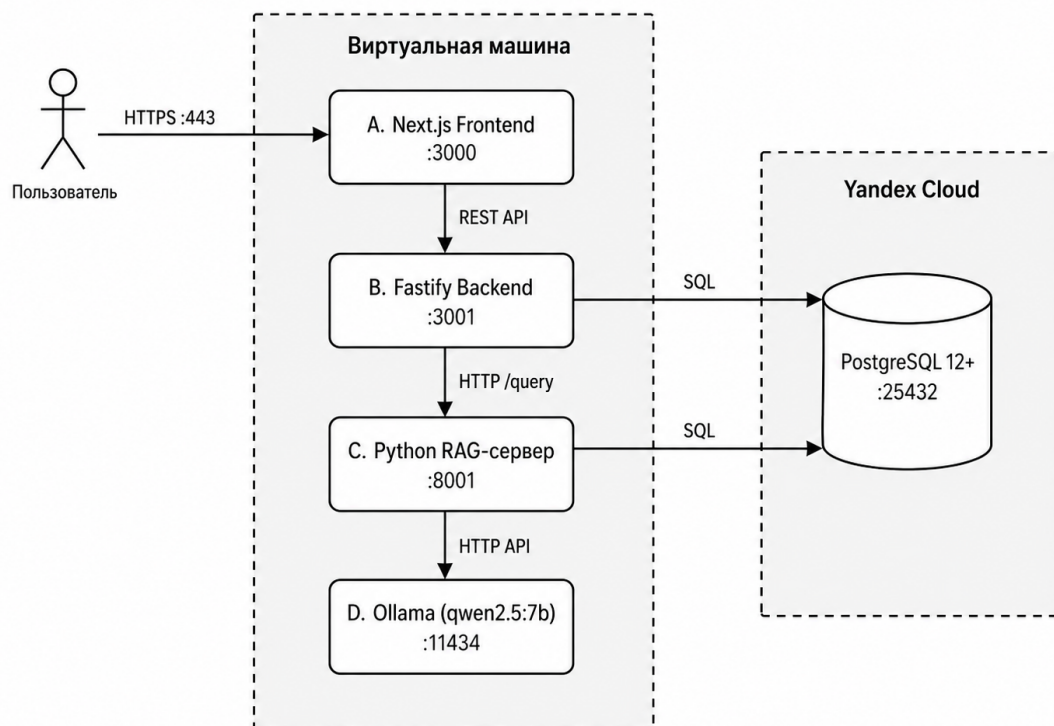


Рис. 4.5: Диаграмма развёртывания системы

Порядок запуска сервисов задаётся в файле `docker-compose.yml`. Сначала должен быть доступен сервис `Ollama`, так как RAG-сервер использует его для генерации ответов. Затем запускается Python RAG-сервер: во время старта он загружает базу знаний, строит embedding-индекс и проверяет подключение к базе данных. После этого запускается основной backend, который подключается к PostgreSQL через пул соединений и получает адрес RAG-сервера из переменной окружения `RAG_SERVER_URL`. Последним становится доступно frontend-приложение, которое обращается к backend по адресу, заданному в `NEXT_PUBLIC_BACKEND_URL`.

Для настройки системы используются переменные окружения. Во frontend задаётся `NEXT_PUBLIC_BACKEND_URL`. Для backend используются параметры подключения к базе данных (`DB_*`), секрет для подписи JWT-токенов (`JWT_SECRET`), адрес RAG-сервера (`RAG_SERVER_URL`), разрешённый origin для CORS, а также параметры `NODE_ENV`, `PORT` и `HOST`. Для RAG-сервера задаются переменные `DB_*`, `OLLAMA_BASE_URL`, `OLLAMA_MODEL`, `EMBEDDING_MODEL`, `RETRIEVAL_TOP_K`, `RAG_SERVER_PORT` и `RAG_SERVER_HOST`. Секретные значения не фиксируются в тексте работы и хранятся отдельно от исходного кода.

В развёртывании предусмотрены базовые меры безопасности. Все запросы от frontend к backend выполняются с использованием JWT-токенов с алгоритмом `HS256`. Пароли пользо-

вателей хранятся в виде хешей, рассчитанных с помощью `bcrypt`. На backend подключается `@fastify/helmet`, добавляющий защитные HTTP-заголовки, включая CSP и HSTS. Также используется `@fastify/rate-limit`, ограничивающий частоту запросов к маршрутам. CORS настраивается только для разрешённого адреса frontend-приложения. При размещении системы на виртуальной машине дополнительно настраивается доступ по протоколу HTTPS и открываются только необходимые порты.

Использование `Docker Compose` упрощает и дальнейшее развитие проекта. При необходимости можно заменить модель в контейнере `Ollama`, изменить параметры RAG-сервера, обновить backend или frontend без ручной настройки всего окружения заново. Кроме того, контейнеризация делает запуск системы воспроизводимым: один и тот же набор сервисов может быть поднят как на локальной машине разработчика, так и на виртуальной машине для демонстрации и тестирования. В дальнейшем такая схема может быть дополнена логированием, резервным копированием базы данных, мониторингом состояния контейнеров и автоматическим перезапуском сервисов при сбоях.

4.7 Выводы по третьей главе

В данной главе было описано проектирование архитектуры разрабатываемого сервиса. По результатам проведённой работы можно сформулировать следующие выводы.

Система построена как совокупность четырёх независимых компонентов: клиентской части, основного backend-сервера, базы данных и RAG-сервера. Такое разделение позволяет изменять каждый компонент без затрагивания остальных и соответствует принципу разграничения зон ответственности при проектировании распределённых систем.

Структура базы данных на основе PostgreSQL охватывает тринадцать таблиц, сгруппированных в три логических блока: справочные данные, операционные сущности бронирования и история диалогов RAG-помощника. Использование перечислений, внешних ключей и индексов обеспечивает целостность данных и производительность при типичных запросах системы.

Клиентская часть на `Next.js 16` организована по компонентному принципу с разделением маршрутов по ролям пользователей. Серверная часть на `Fastify 5` следует многоуровневой архитектуре, где маршруты, контроллеры, сервисы и слой доступа к данным разделены между собой, что упрощает поддержку ролевой модели и управление состоянием заявок.

RAG-модуль реализован как самостоятельный Python-сервис с пайплайном из трёх

шагов: семантический поиск по in-memory индексу на основе модели `paraphrase-multilingual-mpnet-base-v2`, загрузка истории диалога из базы данных и генерация ответа через локальную модель `qwen2.5:7b`. Выбор локального развёртывания языковой модели исключает передачу внутренних документов на внешние серверы.

Развёртывание системы описано через конфигурацию `Docker Compose`, объединяющую все четыре сервиса в единую управляемую среду. Определён порядок запуска компонентов и набор переменных окружения для каждого из них. Базовые меры безопасности — JWT-авторизация, хеширование паролей, защитные HTTP-заголовки и ограничение частоты запросов — реализованы на уровне основного сервера.

Таким образом, спроектированная архитектура обеспечивает разделение основной функциональности бронирования и интеллектуальной справочной поддержки, поддерживает трёхролевою модель доступа и допускает независимое развитие каждого из компонентов системы.

5 Тестирование и оценка разработанного решения

5.1 Подход к тестированию

После завершения разработки системы необходимо убедиться в том, что реализованные компоненты корректно выполняют свои функции и удовлетворяют требованиям, сформулированным на этапе проектирования. Учитывая характер проекта — прикладная информационная система с тремя ролями пользователей и встроенным RAG-модулем — основным подходом к тестированию было выбрано функциональное тестирование на основе пользовательских сценариев. Такой подход позволяет проверить систему не с точки зрения отдельных функций, а с точки зрения целостных действий, которые выполняет реальный пользователь [12].

Тестирование системы выполнялось вручную, так как на данном этапе было важно проверить не только отдельные функции, но и полный путь пользователя внутри сервиса. Проверка была разделена на несколько частей. Сначала рассматривались основные сценарии для трех ролей: менеджера, преподавателя и студента. Для каждой роли проверялось, что пользователь видит нужные разделы, может выполнить предусмотренные действия и не получает доступ к тем функциям, которые ему не предназначены.

Отдельно проверялась работа RAG-помощника. Для этого использовались типовые вопросы, которые могут возникать у участников процесса: вопросы о документах, оплате, правилах ассистирования, сроках и порядке работы с заявками. В данном случае оценивалось

не только то, возвращает ли сервис ответ, но и насколько этот ответ связан с найденными фрагментами документов и не противоречит им.

Третья часть тестирования была связана с ситуациями, в которых пользователь действует некорректно или система получает неправильные входные данные. Проверялись попытки записаться на уже закрытый курс, выбрать количество групп больше доступного, повторно отправить заявку, обратиться к защищенному маршруту без токена или выполнить действие, не соответствующее роли пользователя. Такие проверки нужны для того, чтобы убедиться, что система работает устойчиво не только в правильных сценариях, но и при ошибочных действиях.

Для проверки серверной логики использовались прямые HTTP-запросы к API через Postman. Это позволило тестировать backend независимо от клиентского интерфейса. Такой подход удобен тем, что можно отдельно проверить, как сервер обрабатывает запросы, какие статус-коды возвращает и правильно ли применяет ограничения доступа. В частности, проверялись ответы с кодами 400, 401, 403, 404 и 409, которые соответствуют ошибкам в данных, отсутствию авторизации, запрету доступа, отсутствию ресурса и конфликту состояния.

Также отдельно проверялись ситуации, связанные с безопасностью и ограничением нагрузки. Например, рассматривалось поведение сервера при истечении срока действия JWT-токена, при попытке студента обратиться к маршрутам менеджера, а также при превышении лимита запросов, заданного через `@fastify/rate-limit`. В текущей конфигурации ограничение составляет 100 запросов в минуту. Такая проверка позволила убедиться, что сервер не полагается только на ограничения интерфейса, а самостоятельно контролирует права пользователя и корректность выполняемых операций.

Автоматизированные юнит-тесты и интеграционные тесты в рамках данной работы не разрабатывались, что является осознанным ограничением прототипа. Их написание целесообразно на следующем этапе развития системы, особенно для покрытия бизнес-логики бронирования и проверок доступности групп.

5.2 Проверка пользовательских сценариев

Каждый из трёх ролевых сценариев был последовательно воспроизведён в браузере. Ниже описаны основные сценарии и результаты их проверки.

Сценарий менеджера. Менеджер выполнил вход в систему и перешёл на страницу управления курсами. В форме создания заявки были заполнены все поля: выбран преподаватель из выпадающего списка, введены наименование факультета и программы, выбраны

дисциплина, модули и добавлена учебная ссылка. После отправки формы запись появилась в общем списке заявок. Сервер корректно обработал `upsert`-логику для факультета и программы: при повторном создании заявки с тем же факультетом новая запись в таблице `faculties` не добавлялась. Заккрытие заявки через кнопку удаления переводило её статус в `closed` и убирало из активного списка.

Сценарий преподавателя. После входа преподаватель видел список своих активных заявок, разделённых на группы с назначенным ассистентом и группы со свободными местами. Через страницу поиска ассистентов преподаватель мог просматривать студентов с их профилями, предпочтениями по дисциплинам и количеством активных назначений. Прямое назначение через `POST /api/teacher/assign` создавало бронирование со статусом `active` без дополнительного подтверждения. Принятие входящей заявки студента через `PATCH /api/teacher/bookings/:id/accept` корректно переводило статус с `pending` на `active`.

Сценарий студента. Студент открыл страницу поиска свободных курсов. Фильтрация по преподавателю, дисциплине, программе и модулям работала корректно на стороне клиента без дополнительных запросов к серверу. После выбора курса и указания типа оплаты заявка создавалась со статусом `pending`. Попытка подать повторную заявку на тот же курс возвращала ошибку 409 — ограничение реализовано через уникальный индекс на уровне базы данных. На странице «Мои группы» отображались только активные бронирования с полной информацией о дисциплине, преподавателе и учебных ссылках.

Сценарий RAG-помощника. Все три роли имеют доступ к разделу AI-помощника. При первом обращении без указания `conversation_id` сервер автоматически создавал новую сессию и возвращал её идентификатор вместе с ответом. В последующих обращениях история диалога подгружалась из базы данных и учитывалась при генерации ответа.

Все перечисленные сценарии завершились успешно. Обнаруженные в ходе тестирования несоответствия были исправлены до финальной версии системы.

5.3 Оценка работы RAG-помощника

Для оценки качества RAG-модуля был сформирован размеченный тестовый датасет, включающий 60 вопросов из предметной области. Вопросы составлялись вручную на основе содержимого базы знаний, а также наиболее часто задаваемых вопросов по опыту взаимодействия таким образом, чтобы охватить все тематические разделы документов. Для каждого вопроса был заранее подготовлен эталонный ответ, извлечённый непосредственно из соответ-

ствующего документа. Такой подход позволяет оценить систему независимо от субъективного восприятия качества ответа.

Каждый ответ модели сравнивался с эталонным и относился к одной из трёх категорий: **правильный** — ответ полностью соответствует эталону и не содержит фактических ошибок; **частично правильный** — ответ верен по существу, но неполон или содержит несущественное отклонение; **неправильный** — ответ содержит фактическую ошибку, противоречит документу или модель отказалась отвечать при наличии информации в базе. Итоговая взвешенная точность рассчитывалась как:

$$\text{Accuracy} = \frac{N_{\text{прав}} + 0,5 \cdot N_{\text{частично}}}{N_{\text{всего}}} \quad (2)$$

Результаты тестирования в разбивке по тематическим категориям представлены в таблице 5.1.

Таблица 5.1: Результаты оценки RAG-помощника на размеченном тестовом датасете

Категория вопросов	Всего	Правильных	Частично	Неправильных	Точность
Порядок проведения экзаменов и аттестации	14	11	2	1	85,7%
Апелляции и академические долги	8	6	2	0	87,5%
Спецификации экзаменационных заданий	16	8	7	1	71,88%
Цифровые компетенции	9	6	2	1	77,8%
Шкалы оценок и системы перевода	7	6	1	0	92,9%
Процедурные и ситуационные вопросы	6	4	1	1	75,0%
Итого	60	41	12	4	82,5%

Итоговая взвешенная точность составила 82,5%. Наиболее высокие показатели получены в категориях с чётко структурированными документами: шкалы оценок (92,9%) и апелляционные процедуры (87,5%). В этих разделах документы содержат однозначные числовые данные и таблицы, которые модель воспроизводит точно.

Снижение точности в категориях спецификаций экзаменов (71,88%) и цифровых компетенций (77,8%) объясняется структурой документов: информация о требованиях распределена по нескольким разделам, и при фиксированном значении `retrieval_top_k = 5` часть релевантных фрагментов не попадает в контекст генерации. В четырёх случаях классифицированных как «неправильные» модель либо давала неполный ответ, ссылаясь на отсутствие информации при её фактическом наличии в документах, либо объединяла данные из двух документов с близкими темами некорректным образом.

Ни в одном из 60 случаев модель не генерировала фактически ложную информацию, которой нет в базе знаний, — ограничение через системный промпт и низкое значение температуры (0,2) полностью предотвратили галлюцинации в рамках тестового набора.

Повышение точности в слабых категориях достижимо за счёт увеличения `retrieval_top_k` до 8–10 или введения этапа переранжирования фрагментов (`reranking`). Оба варианта увеличат задержку ответа и нагрузку на языковую модель, что является приемлемым компромиссом при масштабировании системы.

5.4 Сравнение с исходным процессом на основе таблиц

До внедрения разрабатываемой системы процесс согласования назначений ассистентов велся преимущественно через электронную почту и таблицы в Google Sheets. Менеджер формировал список курсов вручную, рассылал его преподавателям, а студенты заявляли о своей готовности через чаты или личные сообщения. Такой подход требовал значительных затрат времени на синхронизацию данных и не исключал конфликтов при одновременной записи.

Разработанная система устраняет основные источники ошибок исходного процесса: дублирование заявок, рассинхронизацию данных между участниками и необходимость ручного контроля статусов. Вместе с тем следует отметить, что внедрение системы требует первоначального времени на регистрацию пользователей и заполнение справочных данных — дисциплин, программ и факультетов.

5.5 Сравнение с существующими аналогами

В рамках НИУ ВШЭ для решения смежных задач существует ряд инструментов, которые студенты и преподаватели используют в своей повседневной работе. К ним относятся: LMS — система управления обучением университета, FAQ-страницы на официальном сайте, нормативный документ ПОПАТКУС (Положение об организации промежуточной аттестации и текущего контроля успеваемости студентов), а также таблицы Excel, применявшиеся для координации назначений ассистентов до разработки настоящей системы.

Для корректного сравнения функциональность оценивалась по двум независимым направлениям: поддержка процесса бронирования и назначения ассистентов, а также возможность получения справочной информации по нормативным документам.

Анализ таблицы показывает следующее. **LMS** обеспечивает управление учебным процессом и поддерживает ролевую модель, однако не содержит функциональности для орга-

Таблица 5.2: Сравнение разработанной системы с исходным процессом

Критерий	Исходный процесс	Разработанная система
Создание заявки на курс	Ручное заполнение таблицы, рассылка по почте	Форма в интерфейсе, мгновенная публикация
Поиск свободных групп студентом	Просмотр общей таблицы, уточнение через чат	Фильтрация по дисциплине, программе и модулям в реальном времени
Подача заявки на ассистирование	Сообщение преподавателю в мессенджере	Кнопка в интерфейсе, статус отслеживается автоматически
Контроль дублирующих заявок	Вручную, высокий риск ошибки	Уникальный индекс в БД, ошибка 409 при повторе
Просмотр актуального статуса группы	Требуется обновления таблицы вручную	Обновляется в реальном времени при каждом запросе
Доступ к справочной информации	Поиск по PDF-документам вручную	RAG-помощник с ответом на естественном языке
Время на создание одной заявки (оценка)	10–15 мин	2–3 мин
Риск конфликта при одновременной записи	Высокий	Исключён на уровне БД
Необходимость администратора для синхронизации	Постоянная	Не требуется

низации бронирования ассистентов и не позволяет вести диалог по нормативным документам. **FAQ-страницы** предоставляют заранее сформулированные ответы на типичные вопросы, но не позволяют задать произвольный вопрос на естественном языке и не охватывают весь объём нормативной документации. **ПОПАТКУС** является исчерпывающим источником правил аттестации, однако для поиска ответа пользователю необходимо самостоятельно ориентироваться в документе объёмом в несколько десятков страниц. **Excel-таблицы** закрывают задачу регистрации назначений, но не защищены от конфликтов при одновременном редактировании и требуют ручной поддержки актуальности.

Разработанная система является единственным инструментом из рассмотренных, который одновременно поддерживает весь цикл назначения ассистентов и обеспечивает интерактивную справочную поддержку на основе документов предметной области. Её очевидным ограничением по сравнению с FAQ остаётся ручной характер обновления базы знаний: при изменении нормативных документов НИУ ВШЭ соответствующие файлы в директории

Таблица 5.3: Сравнение разработанной системы с существующими инструментами

Критерий	Разраб. система	LMS	FAQ	ПОПАТ- КУС	Excel
<i>Процесс назначения ассистентов</i>					
Создание заявки на курс менеджером	+	—	—	—	+
Поиск и фильтрация свободных групп	+	—	—	—	частично
Самостоятельная подача заявки студентом	+	—	—	—	—
Назначение ассистента преподавателем	+	—	—	—	+
Контроль дублирующих заявок	+	—	—	—	—
Актуальность данных в реальном времени	+	—	—	—	—
Ролевое разграничение доступа	+	+	—	—	—
<i>Справочная поддержка</i>					
Поиск по нормативным документам	+	—	частично	—	—
Ответ на естественном языке	+	—	—	—	—
Хранение истории диалогов	+	—	—	—	—
Актуальность базы документов	ручное обновление	—	автомат.	актуален	н/п
Доступность без навыков поиска по PDF	+	—	частично	—	—

knowledge_base/ необходимо обновить вручную и перезапустить RAG-сервер для перестройки индекса. Автоматизация этого процесса через парсинг официального сайта или интеграцию с внутренними системами университета потребует дополнительной инфраструктурной разработки.

5.6 Выводы по четвёртой главе

В данной главе были проверены функциональные характеристики разработанной системы и проведена её сравнительная оценка. По результатам работы можно сформулировать следующие выводы.

Все основные сценарии, которые закладывались в систему, удалось пройти в рабочем

режиме. Менеджер может создать заявку на курс, преподаватель может выбрать ассистента для группы, а студент может найти свободную группу и отправить заявку на ассистирование. Также были проверены ситуации, в которых система должна не просто выполнить действие, а правильно отказать пользователю. Например, при повторной заявке, попытке выбрать недоступное количество групп или обращении к маршруту без нужных прав backend возвращает ошибку и не изменяет данные в базе.

Отдельная часть проверки была связана с RAG-помощником. Для оценки был подготовлен размеченный набор из 60 вопросов по предметной области. По результатам тестирования взвешенная точность составила 82,5%. Лучше всего помощник справился с темами, где документы имеют понятную структуру и ответ обычно находится в одном месте. К таким категориям относятся шкалы оценок, где точность составила 92,9%, и апелляционные процедуры с результатом 87,5%.

Более низкие результаты были получены для вопросов по спецификациям заданий и процедурным ситуациям. В этих категориях точность составила 71,88% и 75,0% соответственно. Основная причина заключается в том, что нужная информация часто распределена между несколькими разделами документа. При фиксированном значении `retrieval_top_k = 5` в контекст модели не всегда попадают все необходимые фрагменты. В дальнейшем эту проблему можно решать увеличением количества извлекаемых фрагментов или добавлением отдельного этапа переранжирования.

Сравнение с исходным процессом показало, что разработанный сервис уменьшает количество ручных действий при создании заявок и снижает риск конфликтов при одновременной записи. В прежнем варианте часть действий выполнялась через таблицы и переписки, поэтому актуальность данных зависела от внимательности участников процесса. В новой системе основные изменения проходят через backend и фиксируются в базе данных, что делает процесс более управляемым.

Сравнение с аналогами также подтвердило необходимость отдельной разработки. Рассмотренные платформы могут закрывать отдельные части задачи: хранение данных, работу с курсами, бронирование времени или справочную поддержку. Однако ни одно из решений не объединяет в одном сервисе ролевую модель бронирования ассистентов, контроль заявок и RAG-помощника, работающего с документами предметной области. Это подтверждает, что специализированный сервис лучше соответствует поставленной задаче.

6 Направления дальнейшего развития

В процессе разработки и тестирования стали заметны несколько направлений, которые могут улучшить систему и сделать ее более удобной для реального использования. Часть из них связана с качеством RAG-помощника, а часть — с дальнейшим развитием самого сервиса бронирования.

Переранжирование результатов поиска в RAG-модуле. Сейчас фрагменты для ответа выбираются по косинусному сходству между вопросом и текстом документа. Такой подход достаточно прост и быстро работает, но не всегда хорошо справляется с вопросами, где ответ разбросан по нескольким разделам. Добавление переранжирования позволило бы сначала находить больше кандидатов, а затем дополнительно выбирать из них наиболее полезные фрагменты. Для этого можно использовать cross-encoder модель, которая оценивает пару “вопрос — фрагмент” более детально. Ожидается, что такой этап особенно поможет в категориях, где текущая точность ниже средней.

Автоматическое обновление базы знаний. В текущей версии документы добавляются в каталог `knowledge_base/`, после чего индекс перестраивается вручную. Для прототипа этого достаточно, но при регулярном использовании такой подход неудобен. В дальнейшем можно реализовать автоматическую загрузку актуальных документов с официальных источников или подключение к внутреннему документообороту. Также полезно добавить версионирование документов, чтобы понимать, какая редакция положения или инструкции использовалась при ответе пользователю.

Интеграция с университетской системой авторизации. Сейчас вход в систему реализован через отдельные учетные записи. В дальнейшем логично подключить университетскую авторизацию, например через OAuth 2.0 или другой корпоративный механизм. Это позволит пользователям входить через уже существующую учетную запись и уменьшит необходимость ручной регистрации. Кроме того, часть данных профиля, таких как факультет, образовательная программа или курс обучения, можно будет подтягивать автоматически, что снизит количество ошибок при заполнении анкеты.

Система уведомлений. На текущем этапе пользователь узнает об изменениях в основном при следующем входе в сервис. Для практического использования удобнее добавить уведомления. Например, студент может получать сообщение о принятии заявки на ассистирование, преподаватель — уведомление о новой заявке от студента, а менеджер — сводку по группам, где ассистент еще не выбран. Уведомления можно реализовать через электронную почту или Telegram-бота, так как эти каналы уже привычны для большинства пользователей.

Автоматизированное тестирование. На текущем этапе тестирование проводилось вручную. Написание интеграционных тестов для ключевых маршрутов API — создания заявки, бронирования, проверки доступности групп — позволило бы выявлять регрессии при дальнейшей разработке. Для серверной части на Fastify это реализуется через встроенный метод `app.inject()`, который позволяет выполнять HTTP-запросы без реального сетевого соединения.

Аналитический дашборд для менеджера. В настоящее время менеджер видит только список активных заявок и статистику заполненности групп по дисциплинам. Расширение этого раздела графиками динамики бронирований, распределением ассистентов по факультетам и историей закрытых заявок дало бы менеджеру инструмент для принятия более обоснованных решений при планировании следующего учебного периода.

Заключение

В рамках данной выпускной квалификационной работы была спроектирована и реализована информационная система для организации процесса назначения студентов-ассистентов на курсы НИУ ВШЭ с интегрированным интеллектуальным помощником на основе технологии RAG.

В ходе работы были достигнуты следующие результаты.

Проведён анализ существующего процесса назначения ассистентов и выявлены его основные недостатки: отсутствие централизованного инструмента, риск конфликтов при одновременной записи, необходимость ручной синхронизации данных между участниками процесса и сложность доступа к нормативной информации. На основании анализа сформулированы функциональные и нефункциональные требования к разрабатываемой системе.

Спроектирована многокомпонентная архитектура, включающая клиентскую часть на Next.js 16 и React 19, основной сервер на Node.js и Fastify 5, реляционную базу данных PostgreSQL из тринадцати таблиц с ролевой моделью, а также отдельный Python-микросервис для обработки запросов к RAG-помощнику. Разделение компонентов обеспечивает независимость их разработки и соответствует принципу разграничения зон ответственности.

Реализован полный цикл бронирования ассистентов: менеджер создаёт заявку на курс, студент подаёт заявку на ассистирование или преподаватель назначает ассистента напрямую, после чего заявка подтверждается или отклоняется. Серверная логика обеспечивает контроль доступности групп, проверку дублирующих заявок и ролевое разграничение до-

ступа на уровне API.

Разработан RAG-модуль на базе модели `paraphrase-multilingual-mpnet-base-v2` и локальной языковой модели `qwen2.5:7b`. База знаний включает 23 документа, охватывающих нормативную документацию НИУ ВШЭ по проведению экзаменов, апелляциям, оценке цифровых компетенций и смежным темам. Тестирование на размеченном датасете из 60 вопросов показало взвешенную точность 82,5%. Ни в одном из тестовых случаев модель не генерировала фактически ложной информации, отсутствующей в базе знаний.

Проведено сравнение разработанной системы с инструментами, применяемыми в университете: LMS, FAQ-страницами, документом ПОПАТКУС и таблицами Excel. Анализ подтвердил, что ни один из существующих инструментов не обеспечивает одновременно полный цикл управления назначениями ассистентов и интерактивную справочную поддержку на основе документов предметной области.

Система развёртывается с использованием `Docker Compose` и включает базовые меры информационной безопасности: JWT-авторизацию, хеширование паролей через `bcrypt`, защитные HTTP-заголовки и ограничение частоты запросов.

Среди направлений дальнейшего развития наибольший практический интерес представляют введение этапа переранжирования фрагментов в RAG-пайплайне, автоматизация обновления базы знаний и интеграция с университетской системой авторизации. Реализация этих улучшений позволит повысить точность ответов помощника и снизить операционные издержки на поддержку системы.

Таким образом, поставленные в работе цели достигнуты: разработанный сервис централизует процесс назначения ассистентов, устраняет основные источники ошибок исходного процесса и предоставляет пользователям удобный инструмент для получения справочной информации без необходимости самостоятельного поиска по нормативным документам.

Список литературы

- [1] Patrick Lewis и др. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. В: *Advances in Neural Information Processing Systems* 33 (2020), с. 9459—9474.
- [2] Ian Sommerville. *Software Engineering*. 9-е изд. Addison-Wesley, 2011. ISBN: 9780137035151.
- [3] Philippe B. Kruchten. “The 4+1 View Model of Architecture”. В: *IEEE Software* 12.6 (1995), с. 42—50. DOI: [10.1109/52.469759](https://doi.org/10.1109/52.469759).
- [4] Sam Newman. *Building Microservices: Designing Fine-Grained Systems*. 2-е изд. O’Reilly Media, 2021. ISBN: 9781492034025.
- [5] C. J. Date. *An Introduction to Database Systems*. 8-е изд. Addison-Wesley, 2003.
- [6] Alex Banks и Eve Porcello. *Learning React: Modern Patterns for Developing React Apps*. 2-е изд. O’Reilly Media, 2020. ISBN: 9781492051725.
- [7] Roy Thomas Fielding. “Architectural Styles and the Design of Network-based Software Architectures”. Дис. ... док. University of California, Irvine, 2000.
- [8] Martin Fowler. *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002. ISBN: 9780321127426.
- [9] Patrick Lewis, Ethan Perez, Aleksandra Piktus и др. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. В: *Advances in Neural Information Processing Systems*. Т. 33. 2020, с. 9459—9474.
- [10] Nils Reimers и Iryna Gurevych. “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks”. В: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*. 2019, с. 3982—3992.
- [11] An Yang и др. “Qwen2.5 Technical Report”. В: *arXiv preprint arXiv:2412.15115* (2024).
- [12] Glenford J. Myers, Corey Sandler и Tom Badgett. *The Art of Software Testing*. 3-е изд. John Wiley & Sons, 2011.

Приложения